

Symmetries in dMod2

Structural identifiability, the observability-identifiability matrix, and symmetry reduction

Simon Beyer

2026-09-20

Contents

1	Introduction	3
1.1	Structural identifiability, local and global	3
2	Symmetries and the determining equations	4
2.1	The infinitesimal generator	4
2.2	Recovering the transformation by integrating the generator	4
2.3	The determining equations	4
2.4	Solving them: the polynomial ansatz	5
3	The observability-identifiability matrix	5
3.1	Output derivatives and the rank criterion	5
3.2	The matrix on a small model	6
3.3	Evaluation over a finite field	6
3.4	Scaling symmetries and large models	7
3.5	Closed-form directions by multi-point reconstruction	7
3.6	Several experimental conditions	8
3.7	Several stimuli in time: piecewise segments	9
3.8	Verifying the rank: the saturation guard	9
4	Supplying the model	10
4.1	Parameter transformations	10
4.2	Steady-state constraints	10
4.3	Free Hill and power exponents	11
5	Removing a symmetry: reduction to invariants	13
5.1	Invariants, and how many are needed	13
5.2	Scalings: an integer kernel	14
5.3	Curved directions: the search	14
5.4	The chart: the old parameters as functions of the new	15
6	Examples	17
6.1	A gene-expression cascade at steady state	17
6.2	Acting on a result: reparametrisation	18
6.3	A phosphorylation cycle, worked through in full	20
6.4	Closed-form directions and a known input	25
6.5	A pre-equilibrated model with a dose	26
6.6	A transient channel made identifiable by exact propagation	28
6.7	An enzyme assay and its hidden symmetries	29
6.8	Two rates on a curve	29

6.9 EGF/EGFR feeding a MEK/ERK cascade	30
References	34

1 Introduction

A parameter is *structurally non-identifiable* when the model produces exactly the same output for a whole family of parameter values, so that no amount of data and no reduction of noise can tell those values apart. This is a property of the model equations and the observation map alone. It is distinct from *practical* non-identifiability, which depends on the data at hand; `symmetryDetection()` addresses the structural question only.

For an ODE model with states $x \in \mathbb{R}^n$ and parameters $\theta \in \mathbb{R}^q$,

$$\dot{x}(t) = f(x(t), \theta), \quad x(0) = x_0(\theta), \quad (1)$$

with observation function

$$g : (x, \theta) \mapsto g(x, \theta) \quad (2)$$

collecting the measured quantities, the question is whether two different unknown vectors produce the same $g(x(t), \theta)$ for all t . If they do, the data constrains only certain combinations of the unknowns.

dMod answers this through the `method` argument of `symmetryDetection()`.

- `method = "observability"` (the default) builds the *observability-identifiability matrix* and reads identifiability off its rank. It uses no ansatz and, for a rational model, is evaluated entirely in exact finite-field arithmetic, so it scales to large systems. With `reconstruct = TRUE` each non-identifiable direction comes back as an exact rational function of the parameters (Section 3.5).
- `method = "polynomial"` searches for *continuous symmetries* via the polynomial Lie-symmetry ansatz and returns the explicit transformation that leaves the output invariant.
- `method = "scaling"` returns the scaling (toric) symmetries $z_i \mapsto \lambda^{c_i} z_i$ exactly, from an integer kernel. Pure integer linear algebra, so it scales, but it sees only the scaling part of the symmetry algebra (Section 3.4).

Detection is half the job: a direction the data cannot constrain has to be removed before fitting, and `symmetryReduction()` does that constructively (Section 5).

Sections 2 and 3 develop the mathematics of the two methods, assuming only multivariable calculus. Section 4 shows how they consume the dMod pipeline objects, and Section 6 works through examples. Section 6.3 carries one model through the whole route, from the determining equations by hand to the reduced parametrisation.

1.1 Structural identifiability, local and global

Collect the unknowns into $z = (x_0, \theta)$ and write $g(t; z) := g(x(t; z), \theta)$. Two unknown vectors are *output-equivalent*, $z \sim z'$, when $g(t; z) = g(t; z')$ for all admissible times and inputs; output-equivalent vectors are indistinguishable from data.

A component z_i is *structurally globally identifiable* if, for generic z ,

$$z \sim z' \implies z_i = z'_i, \quad (3)$$

and *structurally locally identifiable* if (3) holds after restricting z' to a neighbourhood $U \ni z$,

$$z' \in U \text{ and } z \sim z' \implies z_i = z'_i. \quad (4)$$

The model is identifiable when every component is. The two failure modes map onto the two methods:

- A *continuous* family of output-equivalent vectors through z is a local non-identifiability. Its tangent at z is a null vector of the observability-identifiability matrix (Section 3); its explicit form along the family is the symmetry generator (Section 2).
- An *isolated* output-equivalent $z' \neq z$ (a sign flip, say) is purely global and has no infinitesimal signature. Neither method targets it: a full-rank observability matrix certifies *local* identifiability only.

2 Symmetries and the determining equations

2.1 The infinitesimal generator

The unknowns $z = (x_0, \theta) \in \mathbb{R}^{n+q}$ are treated as one augmented system carrying $\dot{\theta} = 0$ alongside (1). A *symmetry* is a smooth, invertible map $z \mapsto \Phi_\varepsilon(z)$ with $\Phi_0 = \text{id}$ that maps every solution of the dynamics to a solution and leaves the output (2) unchanged. The canonical example trades a state against a calibration parameter: if only αx_1 is observed, doubling α and halving x_1 reproduces the readout.

Working with the finite map is awkward; one differentiates at $\varepsilon = 0$ and keeps the *infinitesimals*

$$\eta(z) = \left. \frac{\partial}{\partial \varepsilon} \Phi_\varepsilon(z) \right|_{\varepsilon=0}, \quad X = \sum_{i=1}^{n+q} \eta_i(z) \frac{\partial}{\partial z_i}, \quad (5)$$

the *infinitesimal generator*. For any smooth h ,

$$X(h) = \sum_i \eta_i(z) \frac{\partial h}{\partial z_i} \quad (6)$$

is the rate of change of h along the family, the Lie derivative of h along X . The notation follows Merkt et al. [1], whose algorithm is the "polynomial" method.

2.2 Recovering the transformation by integrating the generator

Both methods return η , not the finite map. The transformation is its flow,

$$\frac{d}{d\varepsilon} \Phi_\varepsilon(z) = \eta(\Phi_\varepsilon(z)), \quad \Phi_0(z) = z, \quad (7)$$

that is $\Phi_\varepsilon = \exp[\varepsilon X]$ (Theorem 1 of [1]).

For a *scaling*, $\eta_i = c_i z_i$, (7) decouples and integrates at once; with $\lambda = e^\varepsilon$,

$$z_i \mapsto \lambda^{c_i} z_i, \quad (8)$$

which is why a scaling is reported by its integer weight vector c : the weights *are* the generator. For a *general* direction the η_i are rational and (7) is a nonlinear coupled flow: still a one-parameter family through z , a curved orbit rather than a ray, but its integral need not be elementary. What the method delivers is always η , exactly.

2.3 The determining equations

Output invariance. With $h = g$ in (6),

$$\sum_{i=1}^{n+q} \eta_i \frac{\partial g_{k'}}{\partial z_i} = 0. \quad (9)$$

Compatibility with the dynamics. Transforming z also transforms $\dot{z}$, so X is *prolonged*,

$$X' = \sum_i \eta_i \frac{\partial}{\partial z_i} + \sum_i \eta'_i \frac{\partial}{\partial \dot{z}_i}, \quad \eta'_i = \sum_j \dot{z}_j \frac{\partial \eta_i}{\partial z_j}, \quad (10)$$

and the model admits X iff $X' \cdot (\dot{z}_k - f_k) = 0$ on solutions (Theorem 2 of [1]), that is

$$\sum_{j=1}^n f_j \frac{\partial \eta_k}{\partial z_j} - \sum_{i=1}^{n+q} \eta_i \frac{\partial f_k}{\partial z_i} = 0, \quad k = 1, \dots, n. \quad (11)$$

The first sum is the change of η_k along f , the second the change of f_k along η ; their cancellation says $[f, X] = 0$, transforming and flowing commute. It runs over the states alone because the parameters do not move under the dynamics, while the second sum runs over all unknowns because f depends on the parameters.

Equations (9) and (11) are the *determining equations*. They are linear in η , and the Lie bracket of two admitted generators is admitted, so the solutions form a Lie algebra: a reported set of directions is a *basis*, not a canonical list.

2.4 Solving them: the polynomial ansatz

η is an unknown *function*, so the system is closed by an ansatz. Merkt et al. [1] take a polynomial one, motivated by the elementary one-parameter groups

$$\begin{aligned} z_i^* &= z_i + \varepsilon, & X &= \partial_{z_i} & (\text{translation}), \\ z_i^* &= e^\varepsilon z_i, & X &= z_i \partial_{z_i} & (\text{scaling}), \\ z_i^* &= \frac{z_i}{1 - \varepsilon z_i}, & X &= z_i^2 \partial_{z_i} & (\text{Möbius}), \end{aligned} \tag{12}$$

whose infinitesimals are all of the form z_i^d , hence

$$\eta_i(z_i) = \sum_{d=0}^{d_{\max}} r_{i,d} z_i^d. \tag{13}$$

Degree $d_{\max} = 1$ covers translations and scalings; higher degrees bring in Möbius transformations and beyond. Letting η_i depend on the other coordinates as well replaces the single power by monomials of total degree $\leq d_{\max}$. The two are `polynomialControl(ansatz = "uni")` and `"multi"`, with `"par"` the intermediate choice, and `pMax = dmax`.

For rational f and g , substituting the ansatz and clearing denominators gives polynomial identities in z ; each monomial coefficient is linear in the $r_{i,d}$, so the system collapses to

$$C r = 0 \tag{14}$$

in $N_r = (n + q)(d_{\max} + 1)$ unknowns, solved in exact rational arithmetic (modular elimination with rational reconstruction), so the rank decision is exact rather than a floating-point threshold, and every surviving generator is verified symbolically. Two sets of rows are added beyond the paper: the Lie chain $X(L_f^k g) = 0$ up to `lieOrder`, and the resting field $[f_{ss}, X] = 0$ when the model is equilibrated.

3 The observability-identifiability matrix

The default method takes a different route to the same information and needs no ansatz. It is the local algebraic observability test of Sedoglavic [2], extended so that the unknowns include parameters and initial states, the FISPO setting of Villaverde et al. [3].

3.1 Output derivatives and the rank criterion

At $t = 0$ the output is g itself; its time derivative follows by the chain rule and (1), $\frac{d}{dt}g|_{t=0} = \sum_i \partial_{x_i} g f_i = L_F g$, and iterating, the k -th derivative is $L_F^k g$. The sequence

$$(g, L_F g, L_F^2 g, \dots, L_F^N g) \tag{15}$$

is the list of Taylor coefficients of the output at $t = 0$, everything the output curve can reveal about the unknowns, and each entry is an explicit function of z (the classical power-series method [4]; used numerically in SIAN [5]).

Collect (15) into $\Phi : z \mapsto (g, L_F g, \dots, L_F^N g)$. Output-equivalent vectors give the same $\Phi(z)$, so the unknowns are locally determined exactly when Φ is injective near a generic point, which holds when its Jacobian

$$\mathcal{O}(z) = \frac{\partial \Phi}{\partial z} = \begin{pmatrix} \partial g / \partial z \\ \partial(L_F g) / \partial z \\ \vdots \\ \partial(L_F^N g) / \partial z \end{pmatrix} \tag{16}$$

has full column rank $n + q$. That is the observability-identifiability matrix (OIM) and the rank condition of Sedoglavic [2]. When the rank is deficient, the null space of $\mathcal{O}$ collects the directions along which the output and all its derivatives are unchanged. These are exactly the values $\eta(z)$ of the generators of Section 2 at z [6]: the two methods describe the same object, one as a tangent vector, the other as a field.

3.2 The matrix on a small model

Take one species with a lumped influx and linear decay, measured directly,

$$\dot{A} = k_1 + u k_2 - k_d A, \quad y = A, \quad (17)$$

with u a known constant input and unknowns $z = (A_0, k_1, k_2, k_d)$. The output and its derivatives are read straight off (17),

$$\begin{aligned} g &= A, \\ L_F g &= k_1 + u k_2 - k_d A, \\ L_F^2 g &= -k_d (k_1 + u k_2) + k_d^2 A, \end{aligned} \quad (18)$$

and differentiating in z at $A = A_0$ gives the first three rows of the OIM,

$$\mathcal{O} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ -k_d & 1 & u & -A_0 \\ k_d^2 & -k_d & -u k_d & -(k_1 + u k_2) + 2k_d A_0 \end{pmatrix}. \quad (19)$$

The two rates enter every derivative only through $K = k_1 + u k_2$, so (19) has a one-dimensional null space spanned by

$$\eta_{k_1} = -u, \quad \eta_{k_2} = 1, \quad \eta_{A_0} = \eta_{k_d} = 0, \quad (20)$$

which one checks row by row ($-u + u = 0$, $u k_d - u k_d = 0$). The higher derivatives add no constraint because they too depend on the rates only through K , which is why the Lie order saturates. A_0 , K and k_d are identifiable. Note that η depends on the input value u : holding u at one generic value reports this trade-off, whereas measuring at two distinct values pins k_1 and k_2 separately (Section 3.6).

3.3 Evaluation over a finite field

Carrying $\mathcal{O}$ symbolically is what makes the naive test intractable: the entries $L_F^k g$ are rational functions whose numerators and denominators grow with every Lie derivative, and the expression swell dominates the cost. The remedy [2] is to evaluate $\mathcal{O}$ at a single generic point, because the rank there equals the symbolic rank away from a measure-zero set.

dMod evaluates in exact arithmetic over $\text{GF}(p)$. Two reasons: every intermediate number stays in $\{0, \dots, p-1\}$, so there is no swell yet the result is exact; and p prime makes $\text{GF}(p)$ a *field*, so every nonzero residue is invertible, which is what row reduction needs. dMod uses primes just below 2^{31} , so products stay within a 63-bit integer.

For a rational model $\mathcal{O}$ is never written symbolically at all. Expanding the solution as $x(t) = \sum_k x_k t^k$ and the right-hand side along it as $f = \sum_k \phi_k t^k$, matching $\dot{x} = f(x)$ order by order gives

$$x_{k+1} = \frac{1}{k+1} \phi_k, \quad (21)$$

where ϕ_k depends only on $x_0, \dots, x_k$ because the series of a product is a convolution $c_k = \sum_i a_i b_{k-i}$ and of a reciprocal the same convolution rearranged. Running the operations that define f on truncated series is automatic differentiation in *Taylor mode*: no symbolic $L_F^k g$ is ever formed, yet the coefficient of t^k in $g(x(t))$ equals $L_F^k g / k!$. The columns come from the same pass by carrying, next to each coefficient, its gradient with respect to z (forward-mode dual numbers nested inside the Taylor recurrence); after the recurrence the gradient parts are the rows of $\mathcal{O}$. The work is a few convolutions per operation and grows only polynomially with model size, so chains that defeat a symbolic construction take a fraction of a second.

Because the construction uses $+, -, \times, \div$ only, it is defined for *rational* right-hand sides and observables; the one exception is a power or Hill term base^n with a parameter exponent, recast to a rational coordinate (Section 4.3). A logarithmic readout $\log_{10}(h) + \text{offset}$ carries the same information as the rational scale $\cdot h$, since a monotone transformation has the same level sets and the additive offset plays the role of a multiplicative

scale, and is supplied in that form. An observable whose *gradient* is not rational either is outside both methods.

One prime and one point already give the rank with probability one, but dMod repeats the evaluation at several points and primes and takes the maximal rank; agreement certifies it, since a non-generic coincidence at every point and prime has probability zero. The points are deterministic, so runs are reproducible. The support of each null vector names the unknowns taking part in the non-identifiability; `reconstruct = TRUE` returns the directions as exact rational functions instead (Section 3.5). The price is that the verdict is *local*, and a null vector gives the tangent rather than the finite transformation; for that, use "polynomial".

3.4 Scaling symmetries and large models

The dominant class of non-identifiability is scaling, $z_i \mapsto \lambda^{c_i} z_i$, whose generator has the constant integer tangent c . Both conditions of Section 2 then become integer-linear: output invariance forces every monomial exponent of the observation to be orthogonal to c , and compatibility forces every monomial of f_i to carry the weight of x_i , that is $(b - e_i) \cdot c = 0$ for each exponent b of f_i . Stacking these and taking the integer kernel returns all scaling symmetries exactly, with no point evaluation and no swell, hence `method = "scaling"` scales to large models, at the price of seeing the scaling part of the algebra only, which in practice is the bulk of identifiability problems (units, calibration, absolute levels).

3.5 Closed-form directions by multi-point reconstruction

The rank verdict needs $\mathcal{O}$ at one point; the *functional* form of a direction needs sampling. Over $\mathbb{Q}(z)$ the null space has a basis whose entries are rational functions of z , and evaluating at one point yields one *sample* of each. Reconstructing a function from samples is interpolation, done here in exact arithmetic.

A canonical sample. At z^* the reduced row echelon form of $\mathcal{O}(z^*)$ over $\text{GF}(p)$ splits the coordinates into pivot and free columns. Each free column f gives one null vector v with $v_f = 1$ and $v_c = -R_{cf}$ on the pivot coordinates. As long as the pivot pattern is generic, the same free column yields the *same* rational functions $r_c(z) := v_c$ at every point, so values from different points are samples of one function.

Two reductions. The scaling directions are peeled off first by the integer kernel of Section 3.4, so only the non-scaling residue is sampled. Then each *entry* is probed for its relevant variables by perturbing each coordinate in turn, and reconstructed over that small set only. Because the probe is per entry, a direction may couple many coordinates while each entry stays narrow, and the cost is set by the widest single entry.

The per-entry fit. Write $r_c(u) = \text{num}(u)/\text{den}(u)$ with total degree at most D . Clearing the denominator gives, at every sample $u^{(s)}$,

$$\text{num}(u^{(s)}) - r_c(u^{(s)}) \text{den}(u^{(s)}) = 0, \quad (22)$$

linear and homogeneous in the unknown coefficients. Stacking over sample points gives a matrix whose kernel is the coefficient vector, up to one common scale that cancels in the ratio. Solving homogeneously, rather than fixing the constant term of den, is what lets the fit recover a denominator without constant term. D is raised from zero: below the true degree the fit is inconsistent, at it the kernel is one-dimensional, and that minimal D is taken; a few extra sample points make a spurious low-degree fit fail (22).

Wide entries. The dense fit costs $\binom{n_{\text{rel}}+D}{n_{\text{rel}}}$ samples, prohibitive for an entry coupling many variables, in practice the conserved-moiety cofactors $-(x_1 + \dots + x_k)/r$. Such an entry is a Laurent polynomial, so on a geometric schedule its values form a linear-recurrence sequence and it is recovered by Ben-Or-Tiwari interpolation [7] from a handful of samples, the work set by the number of terms. An entry with a genuine multi-term denominator is first split by a univariate Cauchy fit along a ray, after which numerator and denominator are ordinary sparse polynomials. Anything beyond the degree caps falls back to its support rather than to a guess.

Lifting $\text{GF}(p)$ back to $\mathbb{Q}$. A single prime returns the coefficients only as residues. The Chinese Remainder Theorem combines residues modulo $p_1, \dots, p_r$ into one value modulo $M = p_1 \dots p_r$, and *rational reconstruction*,

the extended Euclidean algorithm on x and M , turns that integer back into the unique fraction with

$$x \equiv n d^{-1} \pmod{M}, \quad |n|, |d| \leq \sqrt{M/2}. \quad (23)$$

The bound is the point of using several primes: identifiability coefficients are small, so once $\sqrt{M/2}$ exceeds them the lift is exact. dMod uses four primes near 2^{31} , giving $M \approx 2^{124}$ and a range up to $|n|, |d| \approx 2^{61}$.

Certification. Every reconstructed direction is re-evaluated at a fifth prime, disjoint from the four used to build it, and checked against the null vector there; one that fails is reported by its support with `explicit = FALSE` rather than as a wrong closed form. No floating-point number enters anywhere. The caps are the fields of `reconstControl()`, passed as `symmetryDetection(..., control = reconstControl(...))`.

3.6 Several experimental conditions

Each condition c fixes the externally controlled quantities (switches, dose) to known values and may reparametrise a few rates, so it has its own $f^{(c)}$, $g^{(c)}$ and $\mathcal{O}^{(c)}(z)$ over a shared set of unknowns. A direction is invisible in the whole experiment exactly when it is invisible in every condition,

$$\mathcal{O}(z) = \begin{bmatrix} \mathcal{O}^{(1)}(z) \\ \vdots \\ \mathcal{O}^{(K)}(z) \end{bmatrix}, \quad \ker \mathcal{O} = \bigcap_{c=1}^K \ker \mathcal{O}^{(c)}. \quad (24)$$

The intersection can be strictly smaller than any single kernel: a combination hidden under one switch setting is often exposed by another.

This is also the correct treatment of a known switch. Holding a switch at one generic symbol finds the directions valid for *that* generic value and can produce regime-specific artefacts; setting it to its actual value per condition and stacking removes them, and since the switch is then a constant rather than a symbol of z , it cannot appear in any reconstructed direction. Conditions are supplied as an [eventlist] with a condition grid.

3.6.1 Saturation per condition, not on the stack

Raising the Lie order until the rank stops growing is sound for a single condition. With Ω_k the span of $dg, \dots, dL_F^k g$ over the function field, a step that does not grow gives $L_F \Omega_k \subseteq \Omega_k$, since

$$L_F \left(\sum_j c_j dL_F^j g \right) = \sum_j (L_F c_j) dL_F^j g + \sum_j c_j dL_F^{j+1} g, \quad (25)$$

and both sums lie in Ω_k already; by induction no higher order adds anything. One flat step settles it.

The stack (24) inherits none of this. It is a sum of such spans, one per condition and each with its own $F^{(c)}$, and a sum can stand still while a summand grows inside it: a condition with many observables covers for several orders what a sparse one contributes, until the sparse one produces a row outside the sum. Stopping there drops rows, which enlarges $\ker \mathcal{O}$, so the failure mode is again an over-report. The order is therefore saturated per condition and the stack built once, at the largest of them. A stacked rank grows only where some block grows, so this never stops earlier than a plateau taken on the stack, and it is cheaper, the deep orders being reached on one block instead of on all of them.

3.6.2 Specialised coordinates: the codimension budget

The argument also assumes an unspecialised coordinate space. The analysis rarely works with Ω_k itself: an explicit steady state, a pinned initial value or a known parameter pulls it through $J = \partial(x_0, \theta)/\partial z$, and a constraint adds constant rows C , the resting-manifold tangency of `equilibrate` or the relations of a recast exponent. The rank is taken of $W_k = J^* \Omega_k + C$, and with $A = \ker J^*$, $a_k = \dim(\Omega_k \cap A)$ and $c_k = \dim(J^* \Omega_k \cap C)$,

$$\dim W_k = \dim \Omega_k - a_k + \dim C - c_k. \quad (26)$$

Both a_k and c_k are non-decreasing and bounded by $\dim A$ and $\dim C$, so a flat step of W while Ω still grows forces $a_k + c_k$ up, which can happen at most $\text{codim} = \dim A + \dim C$ times over the whole run. A misleading flat step spends one unit of a finite budget.

The rule that follows is to count flat steps cumulatively rather than consecutively, and to stop once more than codim of them have been seen: one was then a genuine flat step, and the rank is final. That is a proof in the specialised space, from the projected curve alone. Without a specialisation the budget is zero and a single flat step certifies, the statement of the previous section. Once the budget is spent, $A \subseteq \Omega_N$ and $C \subseteq J^*\Omega_N$, and (26) makes the projected rank grow exactly where the unprojected one does, so one flat step certifies again.

The budget needs no algebra, only a count: every coordinate of the unspecialised (x_0, θ) space that is not a free coordinate of z contributes at most one, because the rows of J belonging to the untouched coordinates are independent unit vectors. A **trafo** entry, an initial value pinned by the grid, an event or **fixed**, a known parameter and a constraint row each count once, per condition. Started at a steady state the budget is of the order of the state count, so the certificate is cheap on a small model and expensive on a large one. `DMOD_SYM_LIEPLATEAU` caps what is spent, and `summary()` reports which side of the bound a run ended on:

```
Lie order 5 (gap order 0), set by condition 2
saturation: certified (plateau 3 > codimension 2)
```

Below the bound the verdict is provisional and the guard of Section 3.8 is the backstop. The bound is the worst case in $\dim A$: after order N the misleading steps left are $\text{codim} - (\rho_N - r_N)$, where $\rho_N = \dim \Omega_N$ is the rank of the unspecialised system at the same order and $r_N = \dim W_N$ the reported one. That is one further rank on a tape built without the transformation, which the implementation does not compute.

3.7 Several stimuli in time: piecewise segments

An event after t_0 breaks the analytic continuation of the output, so the timeline is split at the distinct event times into segments $S_0, S_1, \dots$, one local jet per segment, stacked exactly as in (24).

The subtlety is the seed of a later segment: building S_j 's jet needs the state *and* its z -Jacobian at the boundary, i.e. the previous segment's state flowed across the gap. That finite-time nonlinear flow is transcendental in the gap length and is not a finite-field number. The exact device is to keep the gap length a **formal power-series variable**: the flow of S_{j-1} over its gap is the state's Taylor series re-read as a series in Δt_{j-1} ,

$$x(\tau_j^-) = \sum_{m \geq 0} x_m \Delta t_{j-1}^m, \quad (27)$$

whose coefficients the kernel already computes (with their z -duals). Every entry of $\mathcal{O}$ becomes a power series in the gaps, and a constant direction v is unobservable for *generic* timing exactly when $R_m v = 0$ for every coefficient layer R_m . Those layers are simply stacked as extra rows: nothing symbolic enters the linear algebra, Δt only labels which layer a row came from. The gap order is raised until the rank stops growing (`info$gapOrderUsed`). This recovers parameters that enter only through a transient between events, an inhibitor pre-incubation say, whose relaxed state at the stimulus is propagated exactly across the gap.

3.8 Verifying the rank: the saturation guard

Where the plateau is shorter than the budget of Section 3.6.2 the verdict rests on a heuristic: the Lie order is raised only until the rank stops growing for a few orders, per condition. A rank can plateau and then climb again, a free Hill exponent and a constrained initial value being the typical cases, and stopping on a false plateau drops rows, which only enlarges $\ker \mathcal{O}$: the failure mode is a silent *over*-reporting. A certified saturation gets no guard, a provisional one does.

`verify = TRUE`, the default, closes that gap, and runs only where the budget did not: it re-uses the kernel and the cached base point and extends the Lie order by `DMOD_SYM_VERIFY_MARGIN` further orders (default six). Only the jet grows; the expensive $f_{ss} = 0$ solve is not repeated. The rank is monotone in the order, so the far end settles the whole window in a single call; the orders in between are built only when that call fails, to

report where the growth starts. If the rank is still climbing, the guard FAILS and warns; if not, saturation is confirmed. `summary()` prints it:

```
saturation guard: PASSED (rank 233 stable through Lie order 18
(6 orders beyond the reported saturation))
```

That rank is *not* the identifiability rank of the header line. The header's rank r / dim d is the rank after projecting onto the d unknowns, so $d - r$ is the number of reported directions; the guard's rank is that of the raw kernel, whose columns span the full jet across all stacked conditions and can be many times d . Only its *constancy* certifies anything. For small models `symEngine = "symbolic"` gives an independent cross-check over the exact rational-function field, with no guard needed.

The guard is complementary to the per-direction certification of Section 3.5: that one rejects a wrong functional *form*, this one checks the *number* of directions.

4 Supplying the model

`symmetryDetection()` needs no compiled model; it takes the equations directly.

argument	meaning
f	the right-hand sides: an <code>eqnlist</code> , an <code>eqnvec</code> , or a named character vector
g	the observation functions, as an <code>eqnvec</code> or named character vector
trafo	an optional parameter transformation (parameter-named entries) and initial conditions (state-named entries)
method	"observability" (default), "polynomial" or "scaling"

Anything `as.eqnvec` accepts is a valid `f`; with an `eqnlist` the conserved moieties are reduced first (`reduceCQ`). Known constants are declared through `fixed`, externally driven inputs through `forcings`, several conditions through `events` and `conditions`, and the pre-stimulus steady state through a steady-state `trafo` or `equilibrate`.

4.1 Parameter transformations

`trafo` is an `[eqnvec]` mapping each inner symbol to an expression in the outer parameters, substituted symbolically into f and g . Three uses occur constantly: reparametrisation (a rate through its half-life); fixing, $k = 1$, which deletes k from the unknowns; and lumping, $k_{\text{deg},m} = k_{\text{deg},p} = k_{\text{deg}}$. A state-named entry is that state's initial condition rather than a substitution, so a solved steady state from `[steadyStates]` can be passed whole.

Structural identifiability is invariant under an invertible, smooth reparametrisation: it neither creates nor removes a non-identifiability, only changes its form. A scaling in natural parameters is a translation in logarithmic ones. Both engines require rationality, so a `trafo` making f or g non-rational, such as $k = \exp(\log k)$, is inadmissible; a log-parametrised model is analysed in its natural rational parameters, with identical conclusions.

4.2 Steady-state constraints

Before the experiment a model often rests at a steady state, so x_0 is not free but solves

$$f_{\text{ss}}(x_0, \theta) = 0, \quad (28)$$

which binds x_0 to θ and can only make the model more identifiable [8]. "polynomial" imposes this as one more commutation condition $[F_{\text{ss}}, X] = 0$. "observability" binds it through the initial condition, in one of two ways: pass a closed form through state-named `trafo` entries (`trafo = steadyStates(reactions)`), or set `equilibrate = TRUE`, which puts the states on a numeric point of (28), recomputed at each evaluation point and prime.

Four things make the latter exact and cheap. The resting system is solved for the *states only*, the rates held as symbolic coefficients, because solving for a rate would express a globally shared rate through condition-specific quantities. States occurring linearly are eliminated by Gaussian elimination and substituted, leaving only the nonlinear core, which over $\text{GF}(p)$ is triangulated by a Groebner basis (grevlex plus FGLM, since a lexicographic basis is ruinous on a coupled core). Degenerate branches where a species vanishes are removed by the Rabinowitsch trick, and the back-substitution accepts a point only when every core equation vanishes, so branches cannot be mixed. Finally, when the resting state is algebraic of degree d its coordinates lie in $\text{GF}(p)$ only at some points and primes, so generic points are drawn until one works for every condition. The parameter sensitivities come from the implicit function theorem, $J_x(\partial x_0/\partial \theta) = -J_\theta$, solved at that point.

The steady state is event-free; perturbations are applied on top of it. A switch is a **forcing**, zero at rest; a dose is a **t0** event composed onto the resting initial condition with its sensitivities. State initial conditions in **trafo** are ignored under **equilibrate**.

4.3 Free Hill and power exponents

A rate with a free exponent is not rational, and is handled by a recast [9]: base^n becomes a coordinate E with companion $L = \log(\text{base})$, carrying

$$\dot{E} = n E \frac{\dot{\text{base}}}{\text{base}}, \quad \dot{L} = \frac{\dot{\text{base}}}{\text{base}}, \quad \frac{\partial E}{\partial \text{base}} = n \frac{E}{\text{base}}, \quad \frac{\partial E}{\partial n} = E L. \quad (29)$$

With $\text{base}^n \rightarrow E$ the field is rational and n enters the OIM through $\partial E/\partial n = E L$, so it is assessed like any other parameter. The logarithm is never evaluated: base , $\log(\text{base})$ and base^n are algebraically independent, so giving L an independent generic residue yields the structural rank by the same argument as Section 3.3.

Without **equilibrate** the base carries a free initial value, so E and L are two further free-initial-value coordinates and the transient itself resolves the exponent. *With* **equilibrate** the base is a solved steady-state coordinate, (28) is solved with E held generic, and the chain rule of E folds into the steady-state Jacobian. (If the base has no linear turnover term in its own balance the roles invert: E is solved rationally and $\text{base} = E^{1/n}$, L are held generic, with duals from the inverse chain rule.)

A self-inhibited producer with a free Hill exponent, observed on an unknown scale and analysed *without* a steady state, has its exponent identified:

```
f <- eqnvec(p = "kpr / (1 + kinh*p^nhill) - dp*p") # free Hill exponent nhill
res <- symmetryDetection(f, eqnvec(y = "s*p"), method = "observability",
                        reconstruct = TRUE, reduceCQ = FALSE)
#> Result: rank 5 / 6 | 1 non-identifiable direction
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> General:
#> X_1
#>      η(kinh) : + kinh*nhill
#>      η(kpr)  : - kpr
#>      η(p)    : - p
#>      η(s)    : + s
c(rank = res$rank, dim = res$dim,
  nhill_identifiable = !("nhill" %in% res$symmetries[[1]]$support))
#>      rank      dim nhill_identifiable
#>      5        6                      1
res$symmetries[[1]]$generator
#> Idx Inner <- Outer
#> 1 kinh <- kinh*nhill
#> 2 kpr <- -kpr
#> 3 p <- -p
#> 4 s <- s
```

Rank 5 of 6: η_{hill} is identifiable and the lone direction is the weighted scaling $\eta_{\text{kinh}} = \eta_{\text{hill}} \cdot \text{kinh}$, $\eta_{\text{kpr}} = -kpr$, $\eta_p = -p$, $\eta_s = s$, rational in η_{hill} with no log term, since here the exponent is pinned rather than freed.

With `reconstruct`, E and L are sampled as coordinates and back-substituted, so a direction that involves the exponent carries an explicit log factor. A feedback species `FB` throttling the production of `x` through $K^n/(K^n + \text{FB}^n)$, with only a scaled `x` measured, shows both regimes at once: the transient resolves the exponent, the resting state does not.

```
hill <- eqnlist() |>
  addReaction("0", "FB", "k_pr_FB") |>
  addReaction("FB", "0", "d_FB * FB") |>
  addReaction("0", "x", "k_pr_x * K^n / (K^n + FB^n)") |>
  addReaction("x", "0", "d_x * x")

# the modular engine assesses the free exponent n through the recast, like any
# parameter. Started from a free initial state, the transient resolves it: n is
# identifiable and the two directions left are ordinary scalings
res <- symmetryDetection(hill, eqnvec(xobs = "scale * x"),
  method = "observability", equilibrate = FALSE,
  reconstruct = TRUE, reduceCQ = FALSE)

#> Result: rank 7 / 9 | 2 non-identifiable directions
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> Scalings:
#> X_1
#>   η(k_pr_x) : + k_pr_x
#>   η(scale)  : - scale
#>   η(x)      : + x
#>
#> X_2
#>   η(FB)      : + FB
#>   η(K)       : + K
#>   η(k_pr_FB) : + k_pr_FB

c(identifiable = res$identifiable, dim = res$dim,
  n_confounded = "n" %in% unlist(lapply(res$symmetries, function(d) d$support)))
#> identifiable      dim n_confounded
#>           0           9           0

# started at rest, that information is gone and n trades against the Michaelis
# constant K with a log factor. The symbolic engine carries base^n as an atom, so
# a free exponent is native to it; it needs the steady state supplied.
ss <- eqnvec(FB = "k_pr_FB/d_FB",
  x = "k_pr_x*K^n/(d_x*(K^n + (k_pr_FB/d_FB)^n))")
sres <- symmetryDetection(hill, eqnvec(xobs = "scale * x"),
  method = "observability", symEngine = "symbolic", trafo = ss)

#> Result: rank 1 / 7 | 6 non-identifiable directions
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> Scalings:
#> X_1
#>   η(K)      : + K
#>   η(d_FB)   : - d_FB
#>
#> X_2
#>   η(K)      : + K
#>   η(k_pr_FB) : + k_pr_FB
#>
```

```

#> General:
#> X3
#>      η(K)      : + (K*K^n + K*(k_pr_FB/d_FB)^n)/(d_x*n*(k_pr_FB/d_FB)^n)
#>      η(d_x)    : + 1
#>
#> X4
#>      η(K)      : + (-K*K^n - K*(k_pr_FB/d_FB)^n)/(k_pr_x*n*(k_pr_FB/d_FB)^n)
#>      η(k_pr_x) : + 1
#>
#> X5
#>      η(K)      : - K*(log(K) - log(k_pr_FB/d_FB))/n
#>      η(n)      : + 1
#>
#> X6
#>      η(K)      : + (-K*K^n - K*(k_pr_FB/d_FB)^n)/(n*scale*(k_pr_FB/d_FB)^n)
#>      η(scale)  : + 1
Filter(function(d) "n" %in% d$support, sres$symmetries)[[1]]$generator
#> Idx Inner <- Outer
#> 1      K <- (-K*log(K)+K*log(k_pr_FB/d_FB))/n
#> 2      n <- 1

```

The exponent direction has $\eta_n = 1$ and

$$\eta_K = \frac{K}{n} \log \frac{k_{pr_FB}/d_{FB}}{K},$$

so at rest the free exponent and the Michaelis constant cannot be told apart (the resting feedback level k_{pr_FB}/d_{FB} is the other base), and the predicted log(base) factor appears explicitly. `equilibrate = TRUE` returns the same verdict from the recast alone. Entered linearly, the same confounder would be an ordinary integer scaling; entered through a Hill term it keeps the exponent in its weight.

5 Removing a symmetry: reduction to invariants

Detection names the free directions. Removing one is a change of parameters, and `symmetryReduction()` constructs it.

It is the construction Lagrangian mechanics performs with generalised coordinates: the configuration space is foliated, by constraint surfaces or by the orbits of a symmetry, and one keeps the coordinates transverse to the leaves and drops the directions along them. Here the space is the parameter space $\mathbb{R}_{>0}^m$, the leaves are the symmetry orbits, and the transverse coordinates are the *invariants*. Three consequences carry over. The count is $\#q = \dim - \dim(\text{leaf})$, here the corank of the observability matrix, whose kernel is tangent to the orbit. The choice is free up to any reparametrisation $q \rightarrow Q(q)$, so the report offers an admissible family instead of one pick. And a generalised coordinate need not inherit an old meaning: θ replaces (x, y) for the pendulum, while a free particle keeps its x . Two things do not carry over. There is no dynamics on the q ; an invariant is a first integral of the flow of X , whose time is the group parameter ε and not the model time t . And the section that picks one point per orbit is a gauge fixing, so it must meet every orbit exactly once, which is what Section 5.4 has to establish.

5.1 Invariants, and how many are needed

For a block of generators $X_1, \dots, X_r$ on m coordinates, $I(z)$ is an *invariant* when

$$X_a(I) = \sum_i \eta_i^{(a)} \frac{\partial I}{\partial z_i} = 0, \quad a = 1, \dots, r, \quad (30)$$

that is when I is constant along every orbit. Output invariance (9) is (30) for $I = g$: the observed quantities are themselves invariants. That is why these are the right coordinates. They are what the data pins down, and the orbit directions are what it cannot.

If the orbits through a generic point have dimension k , the rank of the generator matrix there, then $m - k$ functionally independent invariants coordinatise the orbit space. The reduction is that statement made constructive:

1. find a complete set $I_1, \dots, I_{m-k}$ (Sections 5.2 and 5.3);
2. make them the new parameters;
3. add k equations that single out one point on each orbit, a *section*, and solve for the old coordinates (Section 5.4).

The composite is the emitted `trafo`, ready for `[P()]`.

The naming follows one rule. A scaling leaves each surviving coordinate carrying an invariant monomial, so it keeps its name. A curved carrier is an invariant *value* and not the coordinate it was solved from, so every invariant of a curved block gets a fresh parameter `q_<k>`: reusing a coordinate's name would suggest substitutions that do not exist, a pinned entry `z = 1` fixing the inner coordinate where `q_<k>` is free. The **Invariants**: block of the report gives the invariant behind each `q_<k>`. A scaling survivor is not listed: its monomial follows from the gauge above it, `k_phos_ERK` standing for $k_{phos,ERK} \cdot MEK$ under $MEK = 1$. The full mapping is on the object, as `survivorMeaning`.

5.2 Scalings: an integer kernel

For a scaling block everything is integer linear algebra. Stack the weights as a matrix W , one row per direction, one column per coordinate. A Laurent monomial $z^a = \prod_i z_i^{a_i}$ satisfies (30) exactly when $Wa = 0$, so the invariant monomials are the integer kernel of W .

The section is as explicit. A *transversal* is a set T of k coordinates whose block $A = W[\cdot, T]$ is invertible and for which $M = A^{-1}W$ is an integer matrix, the integrality being what keeps each survivor's monomial Laurent. Pinning T leaves z_j carrying $z_j \prod_{t \in T} z_t^{-M_{tj}}$. Since a scaling orbit is a ray through the positive orthant, the pins may be *any* positive values; 1 is a convention and a known calibration value does as well. The admissible T are the bases of the matroid of the columns of W , and which one to hold is a modelling decision, so the report lists the whole family.

5.3 Curved directions: the search

Beyond scalings there is no lattice. Every stage of the search rests on one observation: (30) is *linear in I* . Each is therefore the same three steps over a richer class of candidates.

1. **Ansatz.** Write I with undetermined coefficients over the class.
2. **Linear system.** Demand $X_a(I) = 0$ identically. By linearity that is a homogeneous system $Cc = 0$, whose rows come from *evaluating* at generic points rather than from symbolic expansion, over $\text{GF}(p)$ with the lift of Section 3.5.
3. **Keep what is new.** Products and powers of invariants are invariants, so the null space is far larger than the number of independent invariants. A Jacobian rank test at a generic point keeps an independent subset, which a certificate such as **2 functionally independent of 9 reports**.

The cheapest class is *Laurent monomials*: for $I = \prod_i z_i^{a_i}$, dividing (30) by I gives

$$\sum_i a_i \frac{\eta_i^{(a)}}{z_i} = 0, \quad (31)$$

linear in the exponent vector a , whose integer null space is the exponent lattice. From there the ladder escalates:

- **Polynomials** of total degree up to `dPoly`. Every monomial coefficient of $X_a(I)$ is a linear form in the unknown coefficients: one row per monomial and generator. Generators with linear components preserve the total degree, so the degrees can be searched one at a time.

- **Separable characteristics.** If every η_i involves no moved coordinate but z_i , the characteristic system $dz_1/\eta_1 = \dots = dz_m/\eta_m$ [10] separates: each ratio integrates on its own as $G_i = \int dz_i/\eta_i(z_i)$, and the differences $G_i - G_j$ are invariants. No degree cap enters and the antiderivative may leave the Darboux language, $\eta = 1 + z^2$ giving $\arctan z$. It runs after the two ansatz stages, which answer in the form one wants to read, $k_1 + u k_2$ rather than $\exp(-(k_1 + u k_2)/u)$.
- **Rational invariants,** through a *Darboux polynomial*: a P with $X(P) = \lambda P$ for a polynomial cofactor λ , which says the surface $\{P = 0\}$ is invariant. Cofactors add under multiplication, so a product of Darboux factors is invariant exactly when its cofactors cancel, and the Darboux-type rational invariants are the integer kernel of the cofactor matrix. Linear once more. The candidates come from the extactic determinant: every Darboux polynomial of degree at most d divides the extactic determinant of the degree- d monomial basis [11], so one determinant and one factorisation deliver all of them.
- **Exponential factors** $\exp(g/h)$, with h a product of Darboux factors and $\deg g \leq \mathbf{dExp}$, each carrying its own cofactor into the same kernel. Here the escalation ends: by Prelle and Singer [12] a vector field with an elementary first integral has one of Darboux form, so within the caps nothing expressible in closed form is missed.
- **Integrating factor,** for a planar block only: an M with $\text{div}(MX) = 0$, then a quadrature. Its value may fall outside the language of a parameter transformation, an $\arctan$ in the damped-oscillator case, and is then reported as an invariant with no reparametrisation claimed.

Before the ladder runs, the block is *module reduced*: over the field of rational functions its generators may be combined with *function* coefficients, which can collapse the support every later stage works in. After it, the count decides. Pinning a coordinate with invariants still missing would discard identifiable structure, so a block that ends short is reported `invariantOnly` with what it found and the caps it exhausted.

5.4 The chart: the old parameters as functions of the new

The invariants define the new parameters. The `trafo` runs the other way, each old coordinate as a function of them. That is one square system, in four steps.

1. **Name.** Each invariant becomes one outer parameter,

$$I_1(z) = q_1, \quad \dots, \quad I_{m-k}(z) = q_{m-k}, \quad (32)$$

which is $m - k$ equations for the m moved coordinates.

2. **Section.** The missing k equations single out one point per orbit. A scaling orbit is a ray and meets $\{z_i = 1\}$ once, so a constant pin suffices. A curved orbit may never reach the pinned value, and may cross it twice and leave the solve without a single branch. `symmetryReduction()` therefore searches for a *balance section*, an equation $m_1 = m_2$ between two low-degree monomials in the moved coordinates, and admits it only if the ratio $r = m_1/m_2$ is strictly monotone along every orbit. That is a sign condition on

$$X(\log r) = \frac{X(m_1)m_2 - m_1X(m_2)}{m_1m_2}, \quad (33)$$

checked symbolically: if the numerator's coefficients have one sign for every generator then r cannot return to a value it has taken, so the section is met exactly once, inside the positive orthant. Mixed signs admit tangencies and are dropped before any solve. With several gauge directions the *set* of sections must be transversal to all of them, which a rank test on the values of (33) decides.

3. **Eliminate.** Equations (32) and the section are m equations in m unknowns, and the elimination is triangular. Each naming equation is solved for one coordinate of its own, its *carrier*, no two invariants sharing one. A carrier v_i is admissible only if that single solve stays inside the language a `trafo` may use: either v_i enters the rational part of I_i in the degrees $\{0, d\}$ only, so that $\text{num} - q_i \text{den} = \alpha v_i^d + \beta$ and v_i follows by a d -th root, with $d = 1$ giving a rational entry, or v_i sits only inside a single exponential factor, linearly in its argument, and follows by a logarithm. Substituting the carriers into the section leaves the k gauge coordinates as the only unknowns; solving for them and back-substituting gives one entry per moved coordinate. Where a carrier's solution still references another carrier, the substitution is iterated until only

outer symbols remain. The degree that survives the substitution fixes the shape of the chart: linear leaves rational entries, a quadratic produces a square root, and a radical denominator is cleared by conjugate multiplication.

4. Certify positivity. Rates and concentrations are estimated on $(0, \infty)$, so the chart is usable only if every entry is positive *for every admissible value of the outer parameters*; otherwise a fit started in the wrong corner leaves the model's domain with no sign that anything is wrong. Admissible is the carrier's own domain: $(0, \infty)$ where the invariant is certified sign-definite on the orthant, $\mathbb{R}$ otherwise. Each entry is therefore proven sign-pure, numerator and denominator of one sign, and an entry $a + c\sqrt{r}$ by the exact comparison of a^2 with $c^2 r$, which also pins the branch of the root. The constant pin $z = 1$ goes through the same test instead of sitting below it as a fallback, so it is emitted exactly where the orbit demonstrably reaches it. If no candidate survives, nothing is emitted.

The four steps decide one geometric object, worth naming because it is easy to confuse with its complement. The image of the emitted trafo is not an orbit; it is the *section*. An orbit has dimension k and carries points the data cannot tell apart, while the image has dimension $m - k$ and carries one representative of every orbit, so on it everything is distinguishable. The two are transverse and their dimensions add up to m . Read that way the certificates are statements about the image: monotonicity in step 2 says each orbit is met at most once, positivity and coverage in step 4 say each is met at least once and inside the positive orthant. Together the image is exactly the reachable part of the section surface, a global cross-section of the map onto the orbit space.

Offsets. Positivity can fail for a reason no section repairs: the invariant itself does not range over all of $(0, \infty)$, so no parameter free there can carry it. The fix is a triangular change of the invariant basis, the outer parameter carrying $I - \varphi$ with the offset φ read off the offending factor of a solved entry, either a constant or a function of the *other* invariants. Two things are proven: $\varphi \geq 0$ for all positive outer values, so the shifted entries stay positive, and *coverage*, $I - \varphi > 0$ on the whole positive orthant with φ composed back into inner coordinates, so the chart still reaches every positive orbit. Coverage is what separates a sharp bound from a lossy one: an offset read off a section that meets only part of the orbit space also yields positive entries, and only coverage rejects it. Section 6.9 carries one through by hand.

Real carriers. An offset needs a finite bound. Where the invariant's value set on the orthant is all of $\mathbb{R}$ there is none, and no reparametrisation $\psi(I)$ inside the language repairs it: a rational ψ that is defined on all of $\mathbb{R}$, injective (hence strictly monotone, so that $\psi(I)$ still separates orbits) and positive does not exist, since $\deg p \leq \deg q$ forces equal limits at $\pm\infty$ while $\deg p > \deg q$ forces $-\infty$ at one end or $+\infty$ at both. The carrier is declared *real* instead, and the gauge follows, a constant pin being reached only by the orbits that happen to cross it. With each entry written as a polynomial in the pin coordinate τ over the invariants,

$$E_i(\iota, \tau) = \sum_{k \leq n_i} a_{i,k}(\iota) \tau^k, \quad a_{i,n_i} > 0, \quad (34)$$

every real root obeys $|\tau| \leq 1 + \sum_k |a_{i,k}/a_{i,n_i}|$, and $|x| \leq (1 + x^2)/2$ turns that into a rational pin

$$\sigma(\iota) = 1 + \sum_i \sum_{k < n_i} \frac{1}{2} \left(1 + (a_{i,k}/a_{i,n_i})^2 \right) \quad (35)$$

above every root of every entry, where each E_i carries the sign of its leading coefficient. For an entry affine in τ this is $\sigma = 1 + \sum_i L_i^2$ with L_i its lower bound, since $1 + L^2 - L = (L - \frac{1}{2})^2 + \frac{3}{4} > 0$. The chart then meets every orbit that meets the orthant, which **coverage** records.

A block can still come back with its invariants and no chart, and that is a statement about the model. An invariant whose *sign* is not constant on the positive orthant, a difference of rates that is a Darboux polynomial and so has a fixed sign along every orbit, is a genuine boundary between components of the orbit space; no chart over free *positive* parameters covers both sides, which is what (35) trades for a real one.

The four steps on the phosphorylation cycle. With the gain s known, the block has the two invariants (51) and (53) on the three moved coordinates (k_p, k_d, P_0) , so one section is needed, and the balance $k_d P = k_p p P$ is monotone along the orbit. Steps 1 and 2 give

$$k_p + k_d = \Lambda, \quad k_p (P_0 + p P_0) = C, \quad k_d P_0 = k_p p P_0, \quad (36)$$

with $p_0 = pP_0$ untouched. The carriers are k_p from the first equation and P_0 from the second, which leaves k_d as the gauge coordinate:

$$k_p = \Lambda - k_d, \quad P_0 = \frac{C}{k_p} - p_0. \quad (37)$$

Substituting both into the section leaves one equation in k_d ,

$$k_d \left(\frac{C}{\Lambda - k_d} - p_0 \right) = (\Lambda - k_d) p_0 \quad \Longleftrightarrow \quad k_d C = \Lambda p_0 (\Lambda - k_d), \quad (38)$$

linear in k_d because the quadratic terms cancel, with the single root $k_d = \Lambda^2 p_0 / (\Lambda p_0 + C)$, the **transversal** line of the report. Back-substituting into (37) gives $k_p = \Lambda - k_d = \Lambda C / (\Lambda p_0 + C)$ and then $P_0 = C / k_p - p_0 = C / \Lambda$, which is the chart (54). Every entry is a ratio of sums of products of $\Lambda, C, p_0 > 0$, so step 4 passes with no offset and the chart covers the whole reachable interval (55). Its image satisfies $k_d P_0 = k_p p P_0$ identically, one codimension inside $(k_p, k_d, P_0, p P_0)$: the section surface, met by each orbit once, and not an orbit itself.

Nothing in the chain is assumed. Every emitted invariant is verified symbolically, $X(I) = 0$ for every generator, before the transformation is assembled, and every entry is proven positive as in step 4. A stage that comes up empty records instead what it excluded, so an unreduced block still says how many invariants the caps produced and therefore which cap to raise; the full per-stage record stays on the object in `$blocks[[i]]$certificates`.

6 Examples

`library(dMod2)`

6.1 A gene-expression cascade at steady state

A stimulus TF drives transcription, the mRNA is translated into protein, both species degrade at a shared rate, and only a scaled protein readout is measured. Before stimulation the cell rests at the basal steady state.

```
eq <- eqnlist() |>
  addReaction("", "mRNA", "ksyn * TF") |>
  addReaction("mRNA", "", "kdeg * mRNA") |>
  addReaction("", "Prot", "ktrl * mRNA") |>
  addReaction("Prot", "", "kdeg * Prot")

g <- eqnvec(yProt = "scale * Prot") # readout of unknown gain

res <- symmetryDetection(eq, g, method = "observability",
  equilibrate = TRUE, reconstruct = TRUE,
  verbose = FALSE)

summary(res) # the report; printing `res` alone shows the directions only
#> -----
#> symmetryDetection | method: observability engine: modular (GF(p) + CRT)
#> -----
#> Computation:
#> Lie order 4 (gap order 0)
#> saturation: certified (plateau 3 > codimension 2)
#> settings: reduceCQ=FALSE, equilibrate=TRUE, reconstruct=TRUE
#> elapsed: 0.1s
#>
#> Result: rank 1 / 5 | 4 non-identifiable directions
#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
```

```

#> X1
#>      η(ksyn) : + ksyn
#>      η(ktrl) : - ktrl
#>
#> X2
#>      η(ktrl) : + ktrl
#>      η(scale) : - scale
#>
#> X3
#>      η(TF)   : + TF
#>      η(ksyn) : - ksyn
#>
#> X4
#>      η(TF)   : + TF
#>      η(kdeg) : + kdeg
#>      η(ktrl) : + ktrl
#>
#> Reduction:
#> X1, X2, X3, X4 are scalings: gauge them by holding one coordinate of each
#> fixed, at any value.

```

At rest the protein sits at $\text{scale} \cdot TF \cdot k_{\text{syn}} k_{\text{trl}} / k_{\text{deg}}^2$, so only that product is observed: rank 1 of 5, four free directions, all exact integer scalings. Three trade the numerator factors of the product; the fourth couples TF , k_{trl} and k_{deg} with equal weight and is induced by the steady state, where the degradation enters squared. Handing the steady state in explicitly with `trafo = steadyStates(eq)` gives the identical verdict.

6.2 Acting on a result: reparametrisation

A non-identifiable direction almost always means that a product or ratio of parameters is identifiable while the factors are not. When a factor is independently known, fix it:

```

res2 <- symmetryDetection(eq, g, method = "observability",
                          equilibrate = TRUE, fixed = "scale",
                          reconstruct = TRUE)
#> Result: rank 1 / 4 | 3 non-identifiable directions
#>
#> Generators X = Σi η(i) ∂i
#>
#> Scalings:
#> X1
#>      η(ksyn) : + ksyn
#>      η(ktrl) : - ktrl
#>
#> X2
#>      η(TF)   : + TF
#>      η(ksyn) : - ksyn
#>
#> X3
#>      η(TF)   : + TF
#>      η(kdeg) : + kdeg
#>      η(ktrl) : + ktrl
c(rank = res2$rank, dim = res2$dim)
#> rank dim
#>    1   4

```

More often the product itself is the meaningful quantity. Carry it as one parameter by fixing the redundant factor to one in a `trafo`:

```

f <- eqnvec(x = "k1*k2 - d*x")           # only the product k1*k2 is identifiable
symmetryDetection(f, eqnvec(y = "x"), method = "observability")$identifiable
#> Result: rank 3 / 4 | 1 non-identifiable direction
#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
#>  $X_1$ 
#>  $\eta(k1) : + k1$ 
#>  $\eta(k2) : - k2$ 
#> [1] FALSE

symmetryDetection(f, eqnvec(y = "x"), method = "observability",
  trafo = eqnvec(k1 = "1", k2 = "k"))$identifiable # now TRUE
#> Result: structurally locally identifiable (rank 3 / 3)
#> [1] TRUE

```

The same recipe resolves any scaling direction: fix all but one of the involved coordinates and let the survivor carry the product.

6.2.1 Automated reduction

`symmetryReduction()` mechanises that and extends it beyond scalings:

```

res <- symmetryDetection(eqnvec(m = "ktx - dm*m", p = "ktl*m - dp*p"),
  eqnvec(y = "p"), method = "observability",
  reconstruct = TRUE, verbose = FALSE)
red <- symmetryReduction(res)
red
#> Reduced 1 of 1 direction.
#>
#> Trafo (non-identity entries)
#>  $ktl = 1$ 
symmetryDetection(eqnvec(m = "ktx - dm*m", p = "ktl*m - dp*p"), eqnvec(y = "p"),
  method = "observability", trafo = red$trafo,
  verbose = FALSE)$identifiable
#> [1] TRUE

```

`print()` shows only the verdict, the trafo and the invariants it is built from; `summary()` adds one line per block, naming its kind, status, stage and how it was gauged, and `summary(verbose = TRUE)` adds the admissible gauges. A candidate fixing of your own goes in through `fixed =`. A transcendental invariant may solve into $\log/\exp$ entries, which `P()` accepts but the rational round trip `symmetryDetection(trafo = )` does not. Directions the search cannot resolve within its degree caps are reported with the caps they exhausted and the remaining options: a structural assumption, a separating experiment, or prediction profiles.

6.2.2 A real-valued carrier

Only $a - b$ enters, and it takes both signs on the orthant, so the carrier is declared real and the pin picks up (35):

```

res <- symmetryDetection(eqnvec(x = "-(a - b)*x"), eqnvec(y = "x"),
  method = "observability", reconstruct = TRUE,
  verbose = FALSE)
symmetryReduction(res)
#> Reduced 1 of 1 direction.
#>
#> Trafo (non-identity entries)
#>  $a = 1 + q_1^2$ 
#>  $b = q_1^2 - q_1 + 1$ 

```

```
#>
#> Invariants:
#>   q_1 = a - b [real-valued]
#>   [real-valued] takes both signs on the positive orthant -- fit it linearly, not on a log scale
```

Both entries are positive for every real q_1 (discriminant $1 - 4 < 0$), so every positive (a, b) is reached; the constant pin $b = 1$ would have reached only $a - b > -1$.

6.3 A phosphorylation cycle, worked through in full

This model is small enough to solve in closed form yet carries both kinds of direction: a scaling that trades amounts against the readout gain, and a curved direction that splits the relaxation rate into its two halves. Everything is derived by hand first and then confirmed. The numerical counterpart, what the directions do to a profile likelihood, is `bench/symmetry_profile_flatness.R` in the package sources.

A protein is phosphorylated at rate k_p and dephosphorylated at rate k_d , and only the phospho-form is measured, on an unknown gain s :

$$\dot{P} = -k_p P + k_d pP, \quad \dot{pP} = k_p P - k_d pP, \quad y = s pP. \quad (39)$$

```
reactions <- eqnlist() |>
  addReaction("P", "pP", "k_p*P", "phosphorylation") |>
  addReaction("pP", "P", "k_d*pP", "dephosphorylation")
observables <- eqnvec(y = "s*pP")
```

The unknowns are $z = (P, pP, k_p, k_d, s)$, $\dim z = 5$; an initial value is carried under the name of its state, so P and pP are P_0, pP_0 .

6.3.1 What the data can see

The cycle conserves $T = P_0 + pP_0$, so the phospho-equation closes on its own, $\dot{pP} = k_p T - \lambda pP$ with $\lambda = k_p + k_d$, and

$$pP(t) = pP_\infty + (pP_0 - pP_\infty) e^{-\lambda t}, \quad pP_\infty = \frac{k_p T}{\lambda}. \quad (40)$$

The measured curve is a single exponential,

$$y(t) = s pP_\infty + s (pP_0 - pP_\infty) e^{-\lambda t}, \quad (41)$$

which carries three coefficients. No experiment on this model can deliver more than

$$\lambda = k_p + k_d, \quad s pP_0, \quad s pP_\infty = \frac{s k_p (P_0 + pP_0)}{k_p + k_d}. \quad (42)$$

Five unknowns, three identifiable functions: the OIM must have rank 3 and two directions must be free. The next section recovers the same verdict from the matrix, which applies equally when no closed form exists.

6.3.2 The generators, from the observability matrix

Write $u = k_p P - k_d pP$ for the net flux, so $\dot{P} = -u$, $\dot{pP} = u$. Differentiating $g = s pP$ along the flow,

$$g = s pP, \quad L_F g = s u, \quad L_F^2 g = -\lambda s u, \quad (43)$$

because $\dot{u} = -\lambda u$: past the first derivative the flow only multiplies by $-\lambda$, so $L_F^k g = (-\lambda)^{k-1} s u$ for $k \geq 1$. The rows of $\mathcal{O}$ are the gradients of (43) in z ,

$$\mathcal{O} = \begin{pmatrix} 0 & s & 0 & 0 & pP \\ s k_p & -s k_d & s P & -s pP & u \\ -\lambda s k_p & \lambda s k_d & -s(u + \lambda P) & -s(u - \lambda pP) & -\lambda u \end{pmatrix}, \quad (44)$$

and every further row is a combination of $d(su)$ and $d\lambda$. The row space is spanned by $d(spP)$, $d(su)$ and $d\lambda$ whatever the Lie order, the rank saturates at 3, and the null space is two-dimensional, the count read off (42), now without the solution. A vector η lies in it exactly when it annihilates the three differentials,

$$\eta_{k_p} + \eta_{k_d} = 0, \quad s\eta_{pP} + pP\eta_s = 0, \quad s(k_p\eta_P - k_d\eta_{pP} + P\eta_{k_p} - pP\eta_{k_d}) + u\eta_s = 0. \quad (45)$$

Any basis of that solution space is an equally valid answer; the natural one splits on whether the gain moves. Taking $\eta_s = -s$ gives $\eta_{pP} = pP$ and, with $\eta_{k_p} = \eta_{k_d} = 0$, $\eta_P = P$:

$$X_1 = P\partial_P + pP\partial_{pP} - s\partial_s. \quad (46)$$

Taking $\eta_s = 0$ forces $\eta_{pP} = 0$, leaves $\eta_{k_d} = -\eta_{k_p}$, and with $\eta_{k_p} = -k_p$ gives $\eta_P = P + pP$:

$$X_2 = (P + pP)\partial_P + k_p\partial_{k_d} - k_p\partial_{k_p}. \quad (47)$$

X_1 is a *scaling* with integer weights ($P: 1$, $pP: 1$, $s: -1$): the amounts enter (42) only through gain-times-amount products. The P -component of X_2 is $P + pP$, not a multiple of P , so X_2 is *general*. It holds s and pP_0 and moves (k_p, k_d, P_0) , which carry only the two constraints λ and $k_p(P_0 + pP_0)$: a fast forward rate on a small unphosphorylated pool cannot be told from a slow one on a large pool.

That split is a choice of reduction, not a taxonomy. X_1 is gauged by holding any one of P , pP , s at any value: its orbit is the ray $(\mu P_0, \mu pP_0, \mu^{-1}s)$, which meets each of $\{P = c\}$, $\{pP = c\}$, $\{s = c\}$ exactly once for every $c > 0$. X_2 has to be reparametrised instead.

```
res <- symmetryDetection(reactions, observables, method = "observability",
                        reconstruct = TRUE)
#> Result: rank 3 / 5 | 2 non-identifiable directions
#>
#> Generators X = \sum_i \eta(i) \partial_i
#>
#> Scalings:
#> X1
#>      \eta(P)   : + P
#>      \eta(pP)  : + pP
#>      \eta(s)   : - s
#>
#> General:
#> X2
#>      \eta(P)   : + P + pP
#>      \eta(k_d) : + k_p
#>      \eta(k_p) : - k_p
```

Rank 3 of 5, and the two directions come back as derived: X_1 under **Scalings** with the weights of (46), X_2 under **General** with the components of (47). The **Reduction** block of **summary(res)** names the two strategies.

6.3.3 The finite transformations

For X_1 the flow is the power law (8), with $\mu = e^\varepsilon$:

$$P_0 \rightarrow \mu P_0, \quad pP_0 \rightarrow \mu pP_0, \quad s \rightarrow \mu^{-1}s. \quad (48)$$

For X_2 the flow decouples in the right order: $dk_p/d\varepsilon = -k_p$ gives $k_p(\varepsilon) = e^{-\varepsilon}k_p$, which drives $dk_d/d\varepsilon = k_p(\varepsilon)$, and $dP/d\varepsilon = P + pP$ integrates with pP fixed. With $\mu = e^{-\varepsilon}$,

$$k_p \rightarrow \mu k_p, \quad k_d \rightarrow k_d + (1 - \mu)k_p, \quad P_0 \rightarrow \frac{P_0 + pP_0}{\mu} - pP_0, \quad (49)$$

with pP_0 and s unchanged: k_d absorbs what k_p gives up so λ is held, and the total moves inversely to k_p so $k_p(P_0 + pP_0)$ is held too. Those are exactly the two constraints of (42) involving the moved unknowns.

Since (41) is available in closed form, the invariance can be checked on the output directly, with no solver and no symbolics:

```

ysol <- function(z, t) {                                     # y(t) from (eq:cycle-sol)
  lambda <- z[["k_p"]] + z[["k_d"]]
  pPinf <- z[["k_p"]] * (z[["P"]] + z[["pP"]]) / lambda
  z[["s"]] * (pPinf + (z[["pP"]] - pPinf) * exp(-lambda * t))
}
flowX1 <- function(z, mu) {                                  # readout units
  z[["P"]] <- mu * z[["P"]]; z[["pP"]] <- mu * z[["pP"]]
  z[["s"]] <- z[["s"]] / mu
  z
}
flowX2 <- function(z, mu) {                                  # split of the relaxation rate
  total <- z[["P"]] + z[["pP"]]
  z[["k_d"]] <- z[["k_d"]] + (1 - mu) * z[["k_p"]]
  z[["k_p"]] <- mu * z[["k_p"]]
  z[["P"]] <- total / mu - z[["pP"]]
  z
}

z <- c(P = 1, pP = 0.2, k_p = 0.3, k_d = 0.05, s = 1.7)
t <- seq(0, 20, by = 0.5)

max(abs(ysol(flowX1(z, 2.5), t) - ysol(z, t)))
#> [1] 8.881784e-16
max(abs(ysol(flowX2(z, 0.4), t) - ysol(z, t)))
#> [1] 4.440892e-16

```

flowX2(z, 0.4) turns $(k_p, k_d, P_0) = (0.3, 0.05, 1)$ into $(0.12, 0.23, 2.8)$, reversing which rate dominates, and the predicted curve does not move.

6.3.4 Finding the invariants

Suppose the readout is calibrated, so s is known. The fixing removes x_1 and leaves the curved x_2 , whose moved coordinates are P , k_p , k_d and whose orbit is one-dimensional: $3 - 1 = 2$ invariants to find, each stage small enough to do by hand.

Laurent monomials. For $I = P^\alpha pP^\beta k_p^\gamma k_d^\delta$, (31) reads

$$\alpha \frac{P + pP}{P} - \gamma + \delta \frac{k_p}{k_d} = (\alpha - \gamma) + \alpha \frac{pP}{P} + \delta \frac{k_p}{k_d} = 0. \quad (50)$$

The functions 1 , pP/P , k_p/k_d are linearly independent, so $\alpha = \gamma = \delta = 0$: no monomial invariant. That is the negative result recorded before the search escalates.

Degree one. With $I = aP + b k_p + c k_d$ (coefficients in the untouched pP , s allowed), $X_2(I) = aP + a pP + (c - b) k_p$, so $a = 0$, $b = c$:

$$I_1 = k_p + k_d. \quad (51)$$

Degree two. One invariant is still missing. X_2 maps the nine degree-two monomials involving a moved coordinate to

$$\begin{array}{lll}
P^2 \mapsto 2P^2 + 2P pP, & P pP \mapsto P pP + pP^2, & P k_p \mapsto pP k_p, \\
P k_d \mapsto P k_d + pP k_d + P k_p, & pP k_p \mapsto -pP k_p, & pP k_d \mapsto pP k_p, \\
k_p^2 \mapsto -2 k_p^2, & k_p k_d \mapsto k_p^2 - k_p k_d, & k_d^2 \mapsto 2 k_p k_d,
\end{array} \quad (52)$$

and collecting $X_2(I)$ by monomial gives a three-dimensional solution space spanned by $k_p (P + pP)$, $pP (k_p + k_d)$ and $(k_p + k_d)^2$. The last two are a product and a square of quantities already known, so the independence

test keeps one new invariant,

$$I_2 = k_p (P_0 + pP_0). \quad (53)$$

With I_1 that is the corank $m - k = 2$, so the search stops at degree two and the Darboux and exponential stages are never reached.

6.3.5 The reduced model

```
red <- symmetryReduction(res, fixed = "s")
summary(red)
#> -----
#> symmetryReduction | from: observability directions: 2
#> -----
#> Reduced 2 of 2 directions.
#>
#> Trafo (non-identity entries)
#>   P   = q_2/q_1
#>   k_d = q_1^2*pP/(q_1*pP + q_2)
#>   k_p = q_1*q_2/(q_1*pP + q_2)
#>
#> Invariants:
#>   q_1 = k_p + k_d
#>   q_2 = k_p*pP + P*k_p
#>
#> Blocks
#> {X1} scaling, fixed | fixed removed 1 direction
#> {X2} general, reduced [polynomial] | section P*k_d = k_p*pP
#>
#> fixed coordinates stay identity entries; keep passing fixed = to downstream
#> calls.
```

The report has one block per coupled set of directions. X_1 and X_2 share the coordinate P , so the scaling is not gauged on its own; it is carried along with the curved direction, and fixing s removes it outright: that is the first block, **fixed**. The second block is X_2 alone, and its stages are the ones computed above. No monomial invariant exists, and the polynomial stage returns (51) and (53), reported as q_1 and q_2 . Write them λ and C .

```
red$trafo
#> Idx Inner <- Outer
#>   1   k_d <- q_1^2*pP/(q_1*pP+q_2)
#>   2   k_p <- q_1*q_2/(q_1*pP+q_2)
#>   3     P <- q_2/q_1
#>   4   pP <- pP
#>   5     s <- s
symmetryDetection(reactions, observables, method = "observability",
                  trafo = red$trafo, fixed = "s", verbose = FALSE)$identifiable
#> [1] TRUE
```

The **trafo** maps the *outer* (estimated) coordinates to the *inner* (model) ones, and the **Invariants:** lines give the outer ones: q_1 is $\lambda = k_p + k_d$, q_2 the flux $C = k_p(P_0 + pP_0)$, and pP keeps its own name. Those are the three quantities of (42) with s known, so the reduced model is parametrised by exactly what the data delivers: three outer parameters where the model had four free coordinates (P_0, pP_0, k_p, k_d) . The redundant direction is gone, not merely constrained. Writing the outer values as (Λ, C, p_0) ,

$$k_p = \frac{\Lambda C}{\Lambda p_0 + C}, \quad k_d = \frac{\Lambda^2 p_0}{\Lambda p_0 + C}, \quad P_0 = \frac{C}{\Lambda}, \quad pP_0 = p_0, \quad (54)$$

and indeed $k_p + k_d = \Lambda$ and $k_p(P_0 + pP_0) = C$ identically.

6.3.6 How far a curved orbit reaches

Parametrise the orbit by k_p ; the invariants fix the rest, $k_d = \lambda - k_p$ and $P_0 = C/k_p - pP_0$, and positivity of the two rates and two amounts bounds

$$0 < k_p < \min\left(\lambda, \frac{C}{pP_0}\right) = \lambda \cdot \min\left(1, \frac{pP_\infty}{pP_0}\right). \quad (55)$$

Both walls mean something: at $k_p = \lambda$ the dephosphorylation has vanished, at $k_p = C/pP_0$ the unphosphorylated pool has. Which binds is decided by the direction of the measured curve: the rate wall for a rising readout ($pP_0 < pP_\infty$), the amount wall for a falling one.

```
mu <- c(0.05, 0.5, 1, 1.15, 1.2) # k_p is multiplied by mu along the orbit
orbit <- t(vapply(mu, function(m) unlist(flowX2(z, m)), numeric(5)))
cbind(orbit[, c("k_p", "k_d", "P")],
      maxdev = apply(orbit, 1, function(r) max(abs(ysol(r, t) - ysol(z, t)))))
#>      k_p    k_d      P    maxdev
#> [1,] 0.015 0.335 23.8000000 4.440892e-16
#> [2,] 0.150 0.200 2.2000000 0.000000e+00
#> [3,] 0.300 0.050 1.0000000 0.000000e+00
#> [4,] 0.345 0.005 0.8434783 0.000000e+00
#> [5,] 0.360 -0.010 0.8000000 0.000000e+00
```

The last row has left the model's domain. The likelihood profiles follow: along the orbit the output does not change at all, so the profile of any coordinate is exactly flat over the interval its orbit reaches and can only rise where the walk must leave the orbit. With $\lambda = 0.35$ here, k_p and k_d are flat below λ , P_0 is flat above $pP_\infty - pP_0 \approx 0.83$, and pP_0 , which no direction moves, is the only coordinate with a proper parabola. Reparametrising with `red$trafo` makes every profile a parabola.

The two walls are different objects, and the distinction is exactly the Darboux bookkeeping of Section 5. The surface $\{k_p = 0\}$ is *invariant*: $X_2(k_p) = -k_p$ vanishes on it, so k_p is a Darboux polynomial with cofactor -1 and no orbit crosses it, so the profile of k_p stays flat all the way down. $\{k_d = 0\}$ is not invariant, since $X_2(k_d) = k_p \neq 0$ there; the orbit crosses at finite ε into $k_d < 0$, outside the model's domain. That crossing, not the symmetry, is the wall: the flatness ends where the model does.

`symmetryReduction()` carries both walls, on request: `reportZeroCompatibility` asks which coordinates the orbit can drive to 0 with nothing running off to infinity – several at once allowed, since a set of rates switched off is still a model. It is off by default: it costs one exact face solve per coordinate of each block.

```
red <- symmetryReduction(res, fixed = "s", reportZeroCompatibility = TRUE)
red$zeroCompatibility[, c("coordinates", "verdict", "limit", "condition")]
#>  coordinates verdict limit    condition
#> 1              P      if FALSE k_d*pP > P*k_p
#> 2             k_d      if FALSE P*k_p > k_d*pP
#> 3             k_p      no      NA
```

k_p is excluded outright – $C = k_p(P_0 + pP_0)$ is invariant and strictly positive, so no orbit carries k_p to zero at finite amounts, which is the Darboux statement again. The other two are the two walls of (55), and `limit = FALSE` says both are met at finite ε . Neither holds everywhere: `condition` is the comparison (55) decides, $P_0 k_p > k_d pP_0$ for the rate wall and its reverse for the amount wall – a rising readout switches the dephosphorylation off, a falling one empties the unphosphorylated pool. That is a property of the point, not of the model, so it is stated as a condition and never as a verdict. The reduction itself takes no parameter values; the condition is an R expression over the model's own names, and one `eval()` settles it wherever it matters:

```
cond <- setNames(red$zeroCompatibility$condition, red$zeroCompatibility$coordinates)
sapply(cond[c("P", "k_d")], function(cc) eval(parse(text = cc), as.list(z)))
#>      P    k_d
#> FALSE TRUE
```

6.3.7 The gauge section

The Family block closes the report with the gauge section $k_d^*P = k_p^*pP$. The naive pin would have failed: by (55) k_d never exceeds λ , so for a slow cycle setting $k_d = 1$ is not a different chart but an unreachable one. Substituting the orbit parametrisation into the section gives a linear equation with the single root

$$k_p^* = \frac{\lambda C}{\lambda p P_0 + C}, \quad (56)$$

the first entry of (54), satisfying both $k_p^* < \lambda$ and $k_p^* < C/pP_0$: the representative lies strictly inside the reachable interval whatever the invariants are. That is the monotonicity condition (33) in this instance.

```
lambda <- z[["k_p"]] + z[["k_d"]]
C      <- z[["k_p"]] * (z[["P"]] + z[["pP"]])
out    <- c(k_p = lambda, P = C, pP = z[["pP"]], s = z[["s"]])      # invariants

inner <- with(as.list(out), c(
  k_p = k_p * P / (k_p * pP + P),
  k_d = k_p^2 * pP / (k_p * pP + P),
  P   = P / k_p,
  pP  = pP,
  s   = s))
signif(inner, 4)
#>      k_p      k_d      P      pP      s
#> 0.29300 0.05698 1.02900 0.20000 1.70000
max(abs(ysol(inner, t) - ysol(z, t)))
#> [1] 0
```

Without the calibration, `symmetryReduction(res)` gauges both directions at once: the block is then $\{X_1, X_2\}$, the invariant set gains spP , and the section becomes $k_d = pP$, $k_p = P$. The reduced model has three free parameters, matching the three numbers of (42).

6.4 Closed-form directions and a known input

The components of a direction may be rational functions of a known input. Model (17) is the minimal case: declaring `u` with `fixed` keeps it out of the unknowns yet lets it appear as a coefficient.

```
f <- eqnvec(A = "k1 + u*k2 - kdeg*A")
toy <- symmetryDetection(f, eqnvec(y = "A"), method = "observability",
  reconstruct = TRUE, fixed = "u")
#> Result: rank 3 / 4 | 1 non-identifiable direction
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> General:
#> X_1
#>      η(k1) : + u
#>      η(k2) : - 1
Filter(function(d) d$type != "scaling", toy$symmetries)
#> [[1]]
#> $type
#> [1] "general"
#>
#> $generator
#> Idx Inner <- Outer
#> 1      k1 <- u
#> 2      k2 <- -1
#>
#> $weights
#> NULL
```

```

#>
#> $degree
#> [1] 1
#>
#> $support
#> [1] "k1" "k2"
#>
#> $explicit
#> [1] TRUE
#>
#> $reason
#> NULL
#>
#> $certified
#> [1] FALSE
#>
#> $transformation
#> NULL
#>
#> $verified
#> [1] NA
#>
#> $display
#> $display$k1
#> [1] "u"
#>
#> $display$k2
#> [1] "-1"
#>
#>
#> attr(,"class")
#> [1] "symmetrygenerator"

```

The direction is the constant $k1 : u$, $k2 : -1$, the input-weighted slope (20) up to sign. When such an input takes several known values across conditions, stacking them separates $k1$ and $k2$ outright, and u , now a constant of each condition, drops out of every reported direction.

6.5 A pre-equilibrated model with a dose

Most experiments do not start from an arbitrary state: the system rests, a stimulus is applied, and the relaxation is recorded. Let

$$\dot{x} = b - ax, \quad y = sx, \quad (57)$$

with basal production b , turnover a and readout scale s , resting at

$$x_* = b/a \quad (58)$$

until a dose sets the level to D at $t = 0$ (a `replace` event), after which

$$x(t) = \frac{b}{a} + \left(D - \frac{b}{a}\right)e^{-at}, \quad y(t) = sx(t). \quad (59)$$

The Taylor coefficients at $t = 0$ are sD , $s(b - aD)$, $sa(aD - b)$, each a multiple of the single relaxation mode with a fixing the ratio between successive coefficients. So a is identifiable, and beyond it the data carry only the two amplitude products sD and sb . With D unknown the OIM has rank 3 and the null space is spanned by $b \mapsto -b$, $s \mapsto s$, $D \mapsto -D$: the products are held but s alone is not. With D set by the experiment, sD pins s , then sb gives b , and the model is identifiable. The pre-equilibration is what makes this clean, since a free initial value would add a second non-identifiability.

```

f <- eqnvec(x = "b - a*x")      # turnover; steady state b/a
g <- eqnvec(y = "s*x")         # scaled readout
dose <- addEvent(eventlist(), var = "x", time = 0, value = "dose",
                 method = "replace")

# a known dose: the relaxation identifies the rate, the scale and the production
symmetryDetection(f, g, method = "observability", events = dose,
                 conditions = data.frame(dose = 2,
                                         row.names = "stim"))$identifiable
#> Result: structurally locally identifiable (rank 3 / 3)
#> [1] TRUE

# an unknown dose magnitude: the readout scale confounds with the amounts
res <- symmetryDetection(f, g, method = "observability", reconstruct = TRUE,
                       events = dose,
                       conditions = data.frame(row.names = "stim"))
#> Result: rank 3 / 4 | 1 non-identifiable direction
#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
#>  $X_1$ 
#>  $\eta(b) : + b$ 
#>  $\eta(dose) : + dose$ 
#>  $\eta(s) : - s$ 
res$symmetries[[1]]$generator
#> Idx Inner <- Outer
#> 1 b <- b
#> 2 dose <- dose
#> 3 s <- -s

```

The direction is the exact integer scaling $b : 1, dose : 1, s : -1$, peeled from the integer kernel, so no reconstruction is needed. The same resting state can be imposed without writing it down:

```

# rank 1 of 3: only the resting product s*b/a is seen, exactly as with the
# explicit steady state trafo = eqnvec(x = "b/a")
res <- symmetryDetection(f, g, method = "observability",
                       equilibrate = TRUE, reconstruct = TRUE)
#> Result: rank 1 / 3 | 2 non-identifiable directions
#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
#>  $X_1$ 
#>  $\eta(b) : + b$ 
#>  $\eta(s) : - s$ 
#>
#>  $X_2$ 
#>  $\eta(a) : + a$ 
#>  $\eta(b) : + b$ 
c(identifiable = res$identifiable, rank = res$rank, dim = res$dim)
#> identifiable      rank      dim
#>      0          1      3

# a genuinely irrational steady state  $x^* = \sqrt{b/a}$ : no rational form exists,
# yet the modular interior point still delivers the verdict
symmetryDetection(eqnvec(x = "b - a*x^2"), g, method = "observability",
                 equilibrate = TRUE)$rank
#> Result: rank 1 / 3 | 2 non-identifiable directions

```

```

#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
#>  $X_1$ 
#>  $\eta(a) : + a$ 
#>  $\eta(b) : - b$ 
#>  $\eta(s) : + s$ 
#>
#> General:
#>  $X_2$  support only
#>  $a, b$ 
#> [1] 1

```

6.6 A transient channel made identifiable by exact propagation

The gap propagation of Section 3.7 identifies parameters that enter only through a transient between events. A species x is produced under a feedback inhibition of strength k_{inh} , switched on by an inhibitor input, and read out on an unknown scale; a stimulus follows later. The inhibition strength affects only the inhibitor-relaxed state at the stimulus onset.

```

f <- eqnvec(x = "kpr/(1 + kinh*inh) - kdeg*x + kstim*stim",
            inh = "0", stim = "0")
g <- eqnvec(y = "s*x")

# inhibitor and stimulus at the same time (no gap): kinh not identifiable
ev0 <- eventlist() |>
  addEvent(var = "inh", time = 0, value = "1", method = "replace") |>
  addEvent(var = "stim", time = 0, value = "1", method = "replace")
r0 <- symmetryDetection(f, g, method = "observability", equilibrate = TRUE,
                      events = ev0, forcings = c("inh", "stim"))

#> Result: rank 3 / 5 | 2 non-identifiable directions
#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
#>  $X_1$ 
#>  $\eta(kpr) : + kpr$ 
#>  $\eta(kstim) : + kstim$ 
#>  $\eta(s) : - s$ 
#>
#> General:
#>  $X_2$  support only
#>  $kinh, kstim$ 
"kinh" %in% unlist(lapply(r0$symmetries, function(d) d$support)) # TRUE
#> [1] TRUE

# inhibitor applied 30 units before the stimulus: the gap is propagated exactly
ev30 <- eventlist() |>
  addEvent(var = "inh", time = -30, value = "1", method = "replace") |>
  addEvent(var = "stim", time = 0, value = "1", method = "replace")
r30 <- symmetryDetection(f, g, method = "observability", equilibrate = TRUE,
                      events = ev30, forcings = c("inh", "stim"))

#> Result: rank 4 / 5 | 1 non-identifiable direction
#>
#> Generators  $X = \sum_i \eta(i) \partial_i$ 
#>
#> Scalings:
#>  $X_1$ 

```

```

#>      η(kpr)   : + kpr
#>      η(kstim) : + kstim
#>      η(s)      : - s
c(gapOrder = r30$info$gapOrderUsed,
  kinhIdentifiable = !("kinh" %in% unlist(lapply(r30$symmetries,
                                                function(d) d$support))))
#>      gapOrder kinhIdentifiable
#>      1          1

```

The second run reports a non-zero `info$gapOrderUsed`, the gap series carrying k_{inh} , and a higher rank than the first.

6.7 An enzyme assay and its hidden symmetries

In the classical enzyme-kinetics assay the non-identifiabilities are not visible by inspection yet can still be derived by hand:

$$\dot{S} = -\frac{k_{\text{cat}} E_{\text{tot}} S}{K_m + S}, \quad y = s S, \quad (60)$$

with unknowns $(S_0, E_{\text{tot}}, K_m, s, k_{\text{cat}})$, $\dim z = 5$. Two substitutions leave the whole trajectory unchanged.

The turnover-enzyme confounder. The pair enters (60) only through $V_{\text{max}} = k_{\text{cat}} E_{\text{tot}}$, so $k_{\text{cat}} \rightarrow \lambda k_{\text{cat}}$, $E_{\text{tot}} \rightarrow \lambda^{-1} E_{\text{tot}}$ fixes everything observed: weights $(k_{\text{cat}} : 1, E_{\text{tot}} : -1)$.

The readout-units confounder. The right-hand side is jointly homogeneous of degree one in the amounts (S, K_m, E_{tot}) , so rescaling them by μ and compensating $s \rightarrow \mu^{-1} s$ keeps $y = sS$: weights $(S : 1, E_{\text{tot}} : 1, K_m : 1, s : -1)$.

Both are scalings, so the data pin exactly sS , sK_m and sV_{max} : three functions, matching a rank of three out of five.

```

f <- eqnvec(S = "-kcat*Etot*S/(Km + S)") # quasi-steady-state turnover
g <- eqnvec(y = "s*S")                  # readout of unknown gain s

res <- symmetryDetection(f, g, method = "observability", reconstruct = TRUE)
#> Result: rank 3 / 5 | 2 non-identifiable directions
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> Scalings:
#> X1
#>      η(Km)   : + Km
#>      η(S)     : + S
#>      η(kcat)  : + kcat
#>      η(s)     : - s
#>
#> X2
#>      η(Etot)  : + Etot
#>      η(kcat)  : - kcat

```

The engine returns two integer scalings spanning the same rank-2 lattice. The printed basis may combine the two hand-derived confounders, since a null space has no canonical basis, but the invariants it fixes are the same.

6.8 Two rates on a curve

A non-identifiability need not be a straight ray. Gene expression observed only at the protein level,

$$\dot{m} = k_{tx} - d_m m, \quad \dot{p} = k_{tl} m - d_p p, \quad y = p, \quad (61)$$

cannot separate transcription from translation: more mRNA poorly translated and less mRNA well translated give the same protein. Scaling the hidden mRNA by any $\lambda > 0$,

$$m \rightarrow \lambda m, \quad m_0 \rightarrow \lambda m_0, \quad k_{tx} \rightarrow \lambda k_{tx}, \quad k_{tl} \rightarrow \lambda^{-1} k_{tl}, \quad (62)$$

leaves $y = p$ invariant. Eliminating λ confines the two rates to $k_{tx} k_{tl} = \text{const}$, a hyperbola rather than a line: only the product, which sets $p_\infty = k_{tx} k_{tl} / (d_m d_p)$, is identifiable. The orbit is curved, but its generator is a plain scaling with weights $(k_{tx}: 1, k_{tl}: -1, m: 1)$:

```
f <- eqnvec(m = "ktx - dm*m", p = "ktl*m - dp*p") # mRNA hidden, protein seen
res <- symmetryDetection(f, eqnvec(y = "p"), method = "observability",
                        reconstruct = TRUE)
#> Result: rank 5 / 6 | 1 non-identifiable direction
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> Scalings:
#> X_1
#>      η(ktl) : + ktl
#>      η(ktx) : - ktx
#>      η(m)   : - m
res$rank # 5 of 6: one direction
#> [1] 5
res$symmetries[[1]]$generator
#> Idx Inner <- Outer
#> 1 ktl <- ktl
#> 2 ktx <- -ktx
#> 3 m <- -m
```

The $m: 1$ weight records that the hidden mRNA scales with k_{tx} along the hyperbola. Degradation rates and the protein do not appear: the data fix them.

6.9 EGF/EGFR feeding a MEK/ERK cascade

A larger but still hand-checkable network shows how to *read* a whole set of directions at once. EGF binds its receptor to form an active complex that phosphorylates MEK, phospho-MEK phosphorylates ERK, and each kinase is returned by a constitutive phosphatase. Only the two phospho-forms are measured, each on an unknown gain, so the ligand, the receptor and the unphosphorylated pools are hidden. Every rate is mass-action, so every direction is rational.

```
reactions <- eqnlist() |>
  addReaction("EGF + EGFR", "EGF_EGFR", "k_bind * EGF * EGFR") |>
  addReaction("EGF_EGFR", "EGF + EGFR", "k_unbind * EGF_EGFR") |>
  addReaction("MEK", "pMEK", "k_phos_MEK * EGF_EGFR * MEK") |>
  addReaction("pMEK", "MEK", "k_dephos_MEK * pMEK") |>
  addReaction("ERK", "pERK", "k_phos_ERK * pMEK * ERK") |>
  addReaction("pERK", "ERK", "k_dephos_ERK * pERK")
reactions <- customTotals(reactions, list(
  totalEGF = "EGF + EGF_EGFR", totalEGFR = "EGFR + EGF_EGFR",
  totalMEK = "MEK + pMEK", totalERK = "ERK + pERK"))
observables <- eqnvec(pMEK_obs = "scale_pMEK * pMEK",
                     pERK_obs = "scale_pERK * pERK")
```

`reduceCQ = FALSE` reports the directions in the species themselves rather than through the moiety totals, which makes them easier to read:

```
egf <- symmetryDetection(reactions, observables, method = "observability",
                        reduceCQ = FALSE, reconstruct = TRUE)
#> Result: rank 11 / 15 | 4 non-identifiable directions
```

```

#>
#> Generators  X = Σi η(i) ∂i
#>
#> Scalings:
#>  X1
#>      η(EGF)      : + EGF
#>      η(EGFR)     : + EGFR
#>      η(EGF_EGFR) : + EGF_EGFR
#>      η(k_bind)   : - k_bind
#>      η(k_phos_MEK) : - k_phos_MEK
#>
#>  X2
#>      η(MEK)      : + MEK
#>      η(k_phos_ERK) : - k_phos_ERK
#>      η(pMEK)     : + pMEK
#>      η(scale_pMEK) : - scale_pMEK
#>
#>  X3
#>      η(ERK)      : + ERK
#>      η(pERK)     : + pERK
#>      η(scale_pERK) : - scale_pERK
#>
#> General:
#>  X4
#>      η(EGF)      : + EGF + EGF_EGFR
#>      η(EGFR)     : - EGFR - EGF_EGFR
#>      η(k_unbind) : - k_bind*(EGF - EGFR)

```

Rank 11 of 15: four directions, three scalings and one rational confounder, each forced by the partial observation.

- **Receptor-module units** ($EGF : 1, EGFR : 1, EGF_EGFR : 1, k_{bind} : -1, k_{phosMEK} : -1$). The receptor module is never seen directly, so its concentration scale is free: multiplying the three species by λ scales the bimolecular flux by λ^2 , undone by $k_{bind} \rightarrow k_{bind}/\lambda$, and the MEK activation is held by $k_{phosMEK} \rightarrow k_{phosMEK}/\lambda$.
- **MEK amount against its readout gain** ($MEK : 1, pMEK : 1, k_{phosERK} : -1, scale_{pMEK} : -1$).
- **ERK amount against its readout gain** ($ERK : 1, pERK : 1, scale_{pERK} : -1$), the same trade-off at the foot of the cascade; nothing lies downstream, so no rate compensates.
- **The binding equilibrium** (general). Only EGF , $EGFR$ and k_{unbind} move, with the complex and k_{bind} held. Because the cascade reads the receptor module only through that complex, the dissociation rate and the split between free ligand and free receptor cannot be told apart.

The scalings integrate by (8) into power laws. The general direction is the same construction with a non-constant η : writing $C = EGF_EGFR$ and $D = k_{bind}(EGF - EGFR)$, the flow (7) is

$$\frac{d EGF}{d\varepsilon} = \frac{-(EGF + C)}{D}, \quad \frac{d EGFR}{d\varepsilon} = \frac{EGFR + C}{D}, \quad \frac{d k_{unbind}}{d\varepsilon} = 1, \quad (63)$$

with C , k_{bind} and everything downstream constant. It has no elementary closed form, but integrating it is unnecessary to see what it does: along the flow

$$\frac{d}{d\varepsilon}(k_{bind} EGF EGFR) = C, \quad \frac{d}{d\varepsilon}(k_{unbind} EGF_EGFR) = C, \quad (64)$$

so binding and unbinding fluxes move in lockstep and their difference, the net rate of complex formation, is preserved. The orbit re-partitions free ligand and receptor and shifts the dissociation rate while holding the active complex and hence the entire measured cascade. That is exactly the non-identifiability the support announced.

6.9.1 Reducing it: the four steps in one report

The receptor-module direction is a scaling, so one of its coordinates may be pinned at any positive value; doing that by hand with `trafo = eqnvec(EGF = "1")` leaves the two readout scalings and the binding equilibrium, and the curved one exercises all of Section 5.4.

```
egf1 <- symmetryDetection(reactions, observables, method = "observability",
  reduceCQ = FALSE, reconstruct = TRUE,
  trafo = eqnvec(EGF = "1"), verbose = FALSE)
red <- symmetryReduction(egf1)
summary(red)
#> -----
#> symmetryReduction | from: observability directions: 3
#> -----
#> Reduced 3 of 3 directions.
#>
#> Trafo (non-identity entries)
#> EGFR = 1
#> EGF_EGFR = (sqrt(q_4 + 1) + 1)/q_4
#> MEK = 1
#> ERK = 1
#> k_bind = q_1*q_4/(sqrt(q_4 + 1) + 1)
#> k_phos_MEK = q_1*q_2*q_4/(sqrt(q_4 + 1) + 1)
#> k_unbind = q_3
#>
#> Invariants:
#> q_1 = EGF_EGFR*k_bind
#> q_2 = k_phos_MEK/k_bind
#> q_3 = EGFR*k_bind - 2*EGF_EGFR*k_bind*sqrt(EGFR/EGF_EGFR + EGFR/EGF_EGFR^2 +
#> 1 + 1/EGF_EGFR) + 2*EGF_EGFR*k_bind + k_bind + k_unbind
#> q_4 = (EGFR*EGF_EGFR + EGFR + EGF_EGFR)/EGF_EGFR^2
#>
#> Blocks
#> {X1} scaling, reduced | transversal MEK = 1, 4 admissible
#> {X2} scaling, reduced | transversal ERK = 1, 3 admissible
#> {X3} general, reduced [exp] | section EGFR = 1
```

The general block moves five coordinates, EGFR, EGF_EGFR, k_bind, k_phos_MEK and k_unbind, along a one-dimensional orbit, so the search looks for $5 - 1 = 4$ invariants. The polynomial stage already meets that count, but one of its members changes sign on the positive orthant and no positive parameter can carry it; the escalation therefore continues and the exponential-factor stage supplies a sign-definite rational replacement, which is why the block reports stage `exp` although the final set is entirely rational:

$$I_1 = C k_{bind}, \quad I_2 = \frac{k_{phosMEK}}{k_{bind}}, \quad I_3 = k_{unbind} + k_{bind}(1 + EGFR), \quad I_4 = \frac{EGFR C + EGFR + C}{C^2},$$

with $C = EGF_EGFR$. Step 2 finds the balance $EGFR \cdot k_{bind} = k_{bind}$ between two summands of I_3 , reported in lowest terms as $EGFR = 1$ and certified monotone along every orbit, as the fifth equation. Step 3 solves the five jointly; the section makes one of them quadratic, hence the $\sqrt{q_4 + 1}$, and step 4 certifies every entry positive for all positive outer values, which is also what pins the branch of that root. I_3 needs an offset, worked out below, after which its entry is the bare parameter, $k_{unbind} = q_3$. Four outer parameters for five moved coordinates, one fewer than the model started with.

The elimination, by hand. Write $C = EGF_EGFR$. On the section $EGFR = 1$ the fourth invariant collapses to $I_4 = (2C + 1)/C^2$, so naming it q_4 gives a genuine quadratic in its carrier,

$$q_4 C^2 - 2C - 1 = 0 \quad \Rightarrow \quad C = \frac{1 \pm \sqrt{1 + q_4}}{q_4},$$

and since $\sqrt{1 + q_4} > 1$ only the $+$ branch is positive: step 4 does the choosing, and the entry is $EGF_EGFR = (\sqrt{q_4 + 1} + 1)/q_4$. The first invariant then gives k_{bind} by division, $k_{bind} = q_1/C$, whose radical

denominator clears by conjugate multiplication, the one rewrite `cancel()` cannot do on its own:

$$k_{bind} = \frac{q_1 q_4}{1 + \sqrt{1 + q_4}} = \frac{q_1 q_4 (\sqrt{1 + q_4} - 1)}{(1 + q_4) - 1} = q_1 (\sqrt{1 + q_4} - 1),$$

and $I_2 = k_{phosMEK}/k_{bind}$ multiplies it up to $k_{phosMEK} = q_2 k_{bind}$.

The third invariant is where an offset becomes unavoidable. On the section $I_3 = k_{unbind} + 2 k_{bind}$, so naming it directly would give $k_{unbind} = q - 2 k_{bind}$, negative wherever q is small: no parameter free on $(0, \infty)$ can carry I_3 . The offset is the AM–GM bound. With $I_4 + 1 = (1 + C)(EGFR + C)/C^2$,

$$\varphi = 2 I_1 (\sqrt{I_4 + 1} - 1) = 2 k_{bind} \left[\sqrt{(1 + C)(EGFR + C)} - C \right],$$

so $I_3 - \varphi > 0$ reads

$$k_{unbind} + k_{bind} [(1 + C) + (EGFR + C)] > 2 k_{bind} \sqrt{(1 + C)(EGFR + C)},$$

which is AM–GM on the two bracketed factors, strict because $k_{unbind} > 0$ – the coverage proof. AM–GM holds with equality exactly when the two factors agree, that is at $EGFR = 1$, which is the section itself; there $\varphi = 2 k_{bind}$ and $I_3 - \varphi$ is *exactly* k_{unbind} . The offset is therefore not merely sufficient but sharp, which is why that entry comes out as the bare parameter, $k_{unbind} = q_3$.

6.9.2 The same network at rest, with a dose

Michaelis–Menten rates, the states started at the resting state `steadyStates()` solves for, and a unit dose of EGF at $t = 0$:

```
reactionsMM <- eqnlist() |>
  addReaction("EGF + EGFR", "EGF_EGFR", "k_bind * EGF * EGFR") |>
  addReaction("EGF_EGFR", "EGF + EGFR", "k_unbind * EGF_EGFR") |>
  addReaction("MEK", "pMEK", "k_phos_MEK * EGF_EGFR * MEK / (Km_MEK + MEK)") |>
  addReaction("pMEK", "MEK", "k_dephos_MEK * pMEK / (Km_pMEK + pMEK)") |>
  addReaction("ERK", "pERK", "k_phos_ERK * pMEK * ERK / (Km_ERK + ERK)") |>
  addReaction("pERK", "ERK", "k_dephos_ERK * pERK / (Km_pERK + pERK)")
rest <- eqnvec(
  k_phos_MEK = "k_dephos_MEK*pMEK*(Km_MEK + MEK)/(EGF_EGFR*MEK*(Km_pMEK + pMEK))",
  k_phos_ERK = "k_dephos_ERK*pERK*(ERK + Km_ERK)/(ERK*pMEK*(Km_pERK + pERK))",
  EGF = "EGF_EGFR*k_unbind/(EGFR*k_bind)"
)dose <- eventlist(var = "EGF", time = 0, value = "1", method = "add")

egf2 <- symmetryDetection(reactionsMM, eqnvec(pMEK_obs = "pMEK", pERK_obs = "pERK"),
  method = "observability", events = dose, trafo = rest,
  reduceCQ = FALSE, reconstruct = TRUE)
#> Result: rank 13 / 14 | 1 non-identifiable direction
#>
#> Generators X = Σ_i η(i) ∂_i
#>
#> General:
#> X_1
#>      η(EGFR)      : + 2*EGFR^2 + 2*EGFR*EGF_EGFR
#>      η(EGF_EGFR) : + EGFR*EGF_EGFR + EGF_EGFR^2
#>      η(k_bind)   : - k_bind*(EGFR + EGF_EGFR)
#>      η(k_unbind) : - EGFR^2*k_bind + EGFR*k_bind + EGF_EGFR*k_unbind
```

A `trafo` right-hand side names *initial values*, never the running states (Section 4.1), and here that distinction is load-bearing: the resting state is written in the state symbols themselves, so reading them as states and substituting $k_{phosMEK}$ back into its own equation gives, with $C = EGF_EGFR$,

$$pMEK \dot{=} k_{phosMEK} \frac{C MEK}{K_{MEK} + MEK} - k_{dephosMEK} \frac{pMEK}{K_{pMEK} + pMEK} \equiv 0, \quad (65)$$

the cascade frozen and every Michaelis constant and phosphatase rate out of the analysis with it. The dose is what makes the resting state informative: without it the trajectory never leaves the steady state and the observation says no more than $pMEK$ and $pERK$ themselves.

Rank 13 of 14, one direction, on the receptor module alone:

$$\eta_{EGFR} = 2 EGFR (EGFR + C), \quad \eta_C = C (EGFR + C), \quad \eta_{k_{bind}} = -k_{bind} (EGFR + C), \quad (66)$$

with $\eta_{k_{unbind}} = k_{bind} EGFR (1 - EGFR) + C k_{unbind}$. Two invariants come straight off (66), since $\eta_{k_{bind}}/k_{bind}$ is both $-\frac{1}{2} \eta_{EGFR}/EGFR$ and $-\eta_C/C$:

$$I_1 = EGFR k_{bind}^2, \quad I_2 = C k_{bind}. \quad (67)$$

The third moves k_{unbind} and comes from the rational stage,

$$I_3 = \frac{EGFR^2 k_{bind} + EGFR k_{bind} + EGFR k_{unbind} + C k_{unbind}}{EGFR} = k_{bind} (EGFR + 1) + k_{unbind} \frac{EGFR + C}{EGFR}. \quad (68)$$

Step 2 balances two summands of (68); in lowest terms the section is $EGFR = 1$, monotone since $X(\log r) = -X(EGFR)/EGFR = -2(EGFR + C) < 0$. Step 4 needs an offset again, and it is AM-GM once more, now between $EGFR$ and 1: $k_{bind}(EGFR + 1) \geq 2 k_{bind} \sqrt{EGFR} = 2\sqrt{I_1}$, so the carrier holds

$$q_3 = I_3 - 2\sqrt{I_1} = k_{bind}(\sqrt{EGFR} - 1)^2 + k_{unbind} \frac{EGFR + C}{EGFR}, \quad (69)$$

a completed square plus a positive term, with infimum 0 – the range a parameter free on $(0, \infty)$ needs. Equality in AM-GM holds exactly at $EGFR = 1$, the section itself, so the offset is sharp there too. It is a function of I_1 alone, which makes $X(q_3) = X(I_3) - 2X(\sqrt{I_1}) = 0$ immediate. The chart then follows from (67) on the section,

$$EGFR = 1, \quad k_{bind} = \sqrt{q_1}, \quad C = \frac{q_2}{\sqrt{q_1}}, \quad k_{unbind} = \frac{\sqrt{q_1} q_3}{\sqrt{q_1} + q_2}, \quad (70)$$

the last entry because $q_3 = k_{unbind}(1 + C)$ there while $\sqrt{q_1} + q_2 = k_{bind}(1 + C)$.

```
summary(symmetryReduction(egf2))
#> -----
#> symmetryReduction | from: observability directions: 1
#> -----
#> Reduced 1 of 1 direction.
#>
#> Trafo (non-identity entries)
#> EGFR = 1
#> EGF_EGFR = q_2/sqrt(q_1)
#> k_bind = sqrt(q_1)
#> k_unbind = sqrt(q_1)*q_3/(sqrt(q_1) + q_2)
#>
#> Invariants:
#> q_1 = EGFR*k_bind^2
#> q_2 = EGF_EGFR*k_bind
#> q_3 = (EGFR^2*k_bind + EGFR*k_bind + EGFR*k_unbind -
#> 2*EGFR*sqrt(EGFR*k_bind^2) + EGF_EGFR*k_unbind)/EGFR
#>
#> Blocks
#> {X1} general, reduced [rational] | section EGFR = 1
```

References

- [1] B. Merkt, J. Timmer, and D. Kaschek, “Higher-order Lie symmetries in identifiability and predictability analysis of dynamic models,” *Physical Review E*, vol. 92, p. 012920, 2015.

- [2] A. Sedoglavic, “A probabilistic algorithm to test local algebraic observability in polynomial time,” *Journal of Symbolic Computation*, vol. 33, pp. 735–755, 2002.
- [3] A. F. Villaverde, A. Barreiro, and A. Papachristodoulou, “Structural identifiability of dynamic systems biology models,” *PLoS Computational Biology*, vol. 12, p. e1005153, 2016.
- [4] H. Pohjanpalo, “System identifiability based on the power series expansion of the solution,” *Mathematical Biosciences*, vol. 41, pp. 21–33, 1978.
- [5] H. Hong, A. Ovchinnikov, G. Pogudin, and C. Yap, “Global identifiability of differential models,” *Communications on Pure and Applied Mathematics*, vol. 73, pp. 1831–1879, 2020.
- [6] G. Massonis and A. F. Villaverde, “Finding and breaking Lie symmetries: Implications for structural identifiability and observability in biological modelling,” *Symmetry*, vol. 12, p. 469, 2020.
- [7] M. Ben-Or and P. Tiwari, “A deterministic algorithm for sparse multivariate polynomial interpolation,” in *Proceedings of the 20th annual ACM symposium on theory of computing (STOC)*, 1988, pp. 301–309.
- [8] M. P. Saccomani, S. Audoly, and L. D’Angiò, “Parameter identifiability of nonlinear systems: The role of initial conditions,” *Automatica*, vol. 39, pp. 619–632, 2003.
- [9] M. A. Savageau and E. O. Voit, “Recasting nonlinear differential equations as S-systems: A canonical nonlinear form,” *Mathematical Biosciences*, vol. 87, pp. 83–115, 1987.
- [10] P. J. Olver, *Applications of Lie groups to differential equations*, 2nd ed. New York: Springer, 1993.
- [11] C. Christopher, J. Llibre, and J. V. Pereira, “Multiplicity of invariant algebraic curves in polynomial vector fields,” *Pacific Journal of Mathematics*, vol. 229, pp. 63–117, 2007.
- [12] M. J. Prele and M. F. Singer, “Elementary first integrals of differential equations,” *Transactions of the American Mathematical Society*, vol. 279, pp. 215–229, 1983.